MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND
COMMUNICATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING

GRADUATION THESIS
BACHELOR OF ENGINEERING IN
INFORMATION TECHNOLOGY

(HIGH-QUALITY PROGRAM)

BUILDING A FASHION E-COMMERCE APPLICATION WITH IMAGE-BASED PRODUCT SEARCH AND MODEL BENCHMARKING

Student: Nguyen Thanh Phat
Student ID: B2005853
Class: DI20V7F1 (K46)
Advisor: Dr. Tran Cong An

Can Tho, December 2025

MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING

GRADUATION THESIS
BACHELOR OF ENGINEERING IN INFORMATION TECHNOLOGY

(HIGH-QUALITY PROGRAM)

BUILDING A FASHION E-COMMERCE APPLICATION WITH IMAGE-BASED PRODUCT SEARCH AND MODEL BENCHMARKING

Student: Nguyen Thanh Phat
Student ID: B2005853
Class: DI20V7F1 (K46)
Advisor: Dr. Tran Cong An

Can Tho, 01/2026

EVALUATION OF ADVISOR

Advisor:

Dr. Tran Cong An

ACKNOWLEDGEMENTS

I wish to express my deep gratitude to my advisor for their guidance throughout this thesis. I would also like to thank the lecturers at Can Tho University for their invaluable knowledge.

I am grateful to my family for their love and support, and to my friends for their encouragement.

Sincerely,

Can Tho, [Date]

[Your Name]

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
ANN	Approximate Nearest Neighbor
CBIR	Content-Based Image Retrieval
CNN	Convolutional Neural Network
CQRS	Command Query Responsibility Segregation
DDD	Domain-Driven Design
DSR	Design Science Research
EF Core	Entity Framework Core
HNSW	Hierarchical Navigable Small World
JWT	JSON Web Token
mAP	Mean Average Precision
P@K	Precision at K
R@K	Recall at K
REST	Representational State Transfer
SPA	Single Page Application
ViT	Vision Transformer
VSA	Vertical Slice Architecture

ABSTRACT

E-commerce platforms rely primarily on text-based search, yet fashion products are inherently visual — patterns, silhouettes, and textures resist keyword description. Consumers can easily recognize a specific garment by its visual appearance but struggle to articulate it using standardized metadata terms. This thesis presents a fashion e-commerce platform with integrated Content-Based Image Retrieval that enables customers to search for products by uploading images rather than typing keywords. The system implements a modular architecture with a .NET backend, Vue.js frontend, and a Python machine learning sidecar for embedding generation, connected through a service-oriented design that bridges enterprise-grade web application reliability with access to the Python artificial intelligence ecosystem.

A systematic benchmark evaluates eleven pre-trained deep learning models spanning convolutional neural networks — ResNet and EfficientNet variants — and vision transformers — DINOv2 and CLIP-based architectures including a fashion-specific fine-tuned variant — across both retrieval accuracy and operational efficiency. The evaluation framework measures mean Average Precision, Precision at K, Recall at K, inference latency, throughput, and resource consumption across a curated fashion product dataset.

Results demonstrate that fashion-specific models achieve measurable advantages for visual fashion retrieval, with domain-trained embeddings consistently outperforming general-purpose counterparts in retrieval quality while maintaining inference speeds viable for real-time deployment on commodity hardware. The work shows that open-source tools combined with a pluggable model architecture can deliver production-grade visual search capabilities comparable to commercial solutions, while providing a reference implementation for systematic comparison of embedding models in the fashion domain.

Keywords: e-commerce, visual search, deep learning, modular architecture, computer vision, benchmarking

PART 1: INTRODUCTION

2 INTRODUCTION

2.1 CONTEXT AND MOTIVATION

The discovery and purchase of clothing have shifted fundamentally toward digital platforms, yet the methods for locating specific products remain largely tethered to text-based retrieval. Global fashion e-commerce revenue exceeded 770 billion U.S. dollars in 2024 and is projected to surpass one trillion by 2030 [1], positioning the sector among the fastest-growing segments of online retail. Despite this expansion, the search bar — which serves as the primary interface between users and product catalogs — often struggles to interpret the visual nuances that define fashion. This limitation is central to the motivation of this project.

A persistent challenge in this domain is the **semantic gap**: the discrepancy between the visual complexity of a product and the linguistic capability of a user to describe it. A customer may easily recognize a specific pattern, silhouette, or texture but struggle to articulate it using standardized metadata terms like "bohemian asymmetric maxi dress with botanical motifs." If the product catalog's indexing does not perfectly align with the user's vocabulary, the search fails despite the product's existence. Studies on e-commerce search behavior indicate that up to 30 percent of shoppers abandon a site after an unsuccessful search [2], reflecting the commercial cost of this gap. The problem is particularly acute in fashion, where visual attributes — drape, proportion, colour gradients, and texture — resist reduction to keywords, and where the same garment is described differently by different sellers, different brands, and different customers.

Modern visual search systems, powered by deep learning, offer a path beyond keyword limitation. Instead of relying on human-assigned labels, these systems encode the visual content of images into dense vector representations — embeddings — that can be compared mathematically for similarity. A query image of a dress with a particular neckline and print pattern can retrieve visually similar products without any textual intermediary. This capability, known as Content-Based Image Retrieval (CBIR), has been accelerated by advances in pre-trained convolutional neural networks [3], [4] and vision transformers [5], including fashion-specific models trained on domain-relevant data [6].

The central question this thesis investigates is not whether visual search works — that has been demonstrated in both academic literature and industrial deployments — but rather **how** to integrate these capabilities into a practical e-commerce system built with conventional web technologies, and **which** embedding models deliver the best balance of accuracy, speed, and resource efficiency in that context. This is fundamentally an engineering problem: the spanning of two distinct software ecosystems — the Python machine learning stack and the .NET enterprise web stack — while serving real-time user requests within latency budgets acceptable for interactive search.

2.2 PROBLEM STATEMENT

The core problem addressed by this project is the inherent inefficiency of keyword-reliant search for fashion discovery. This broad challenge is decomposed into four concrete technical and functional issues:

Linguistic inconsistency. Traditional search systems depend on precise, consistent product labels. Large-scale catalogs, however, frequently suffer from descriptor variance: one listing uses "floral," another "botanical," a third "flower print," all referring to the same visual property. This inconsistency fragments search results, surfaces incomplete matches, and forces users to iterate through multiple query formulations to find what they seek.

Visual inexpressibility. Many defining fashion attributes — draping, texture, colour gradients, silhouette shape, and pattern density — are intuitive to the human eye but difficult to translate into text queries. A customer can identify a desired aesthetic in a photograph but cannot produce the keywords that would retrieve items matching that aesthetic. This mismatch leads to high bounce rates and lost conversions.

Cold start data scarcity. Recommendation systems frequently rely on collaborative filtering, which aggregates historical user-item interactions to surface relevant products. New items, by definition, lack this interaction data. Visual feature extraction offers an alternative path: by encoding product images into embeddings, similarity-based recommendations become possible for newly listed inventory based solely on appearance, without any interaction history.

Polyglot integration complexity. The deep learning ecosystem — frameworks such as PyTorch, the HuggingFace model hub, and the broader Python scientific computing stack — does not natively interoperate with the .NET ecosystem used for transactional e-commerce backends. Bridging these two environments in a production-capable system requires careful architectural design to ensure low latency, fault isolation, and maintainable inter-service communication. This thesis addresses this challenge through a modular monolith architecture augmented with a dedicated Python sidecar service, avoiding

the operational overhead of a full microservices deployment while preserving the benefits of technology-appropriate service boundaries.

2.3 OBJECTIVES

The primary goal of this project is to build a functional fashion e-commerce platform that enables image-based product search, and to evaluate how effectively pre-trained deep learning models can be integrated into such a system. Rather than developing novel AI architectures, this work concentrates on the engineering challenge of embedding existing models into a practical web application stack.

2.3.1 Technical Objectives

This project addresses specific engineering challenges through the following objectives:

- **Demonstrating the integration** of pre-trained deep learning models into a conventional e-commerce stack built on PostgreSQL and .NET, establishing a reference pattern for teams with existing web infrastructure who wish to add visual search capabilities.
- **Architecting a polyglot system** that efficiently bridges .NET transactional logic with Python-based AI inference, using a sidecar service pattern that isolates the machine learning workload while maintaining low-latency communication with the main application.
- **Validating the feasibility** of open-source vector database tools — specifically the pgvector extension for PostgreSQL — for real-time similarity search at a scale representative of small-to-medium fashion catalogs.
- **Benchmarking the performance** of multiple embedding models spanning convolutional neural network and vision transformer architectures within a constrained hardware environment, providing empirical guidance for practitioners selecting models for similar deployments.

2.3.2 Research Questions

The project aims to answer the following questions:

1. **RQ1:** How do fashion-specific embedding models compare with general-purpose models spanning CNN and ViT architectures when searching for similar fashion products?
2. **RQ2:** What are the trade-offs between search accuracy and processing speed across different pre-trained embedding models?
3. **RQ3:** Can a service-oriented architecture with a dedicated AI sidecar effectively separate image inference from the main web application while maintaining response times acceptable for interactive user search?

2.3.3 Specific Tasks

To answer these questions, the following tasks were completed:

1. **Build an AI service.** Develop a Python service using FastAPI that loads and executes multiple pre-trained image embedding models. The service accepts product images, generates feature vectors, and returns results within a target latency budget appropriate for interactive use.
2. **Set up vector search.** Configure PostgreSQL with the pgvector extension to store and index high-dimensional image embeddings. Validate that similarity search queries execute correctly and efficiently on a catalog scale representative of a small-to-medium fashion retailer.
3. **Connect the services.** Implement a .NET backend layer that communicates with the Python AI service via HTTP, orchestrating the end-to-end search flow — image upload, feature extraction, vector database query, and result return — within the latency envelope expected by end users.
4. **Create the user interface.** Build a Vue.js storefront application where users can upload query images via drag-and-drop or file selection and browse visually similar products returned by the search pipeline.
5. **Evaluate the results.** Conduct a systematic benchmark measuring retrieval accuracy through standard information retrieval metrics, comparing processing speed across models, and analyzing the operational trade-offs that inform model selection for production deployment.

2.4 SCOPE AND LIMITATIONS

Included Scope:
- Image upload and visual search functionality accessible from the storefront user interface
- Product recommendations derived from visual similarity
- Core e-commerce features, including product catalog browsing and shopping cart management
- Systematic comparison of multiple embedding models spanning both CNN-based and transformer-based architectures
- End-to-end performance measurement covering inference latency, query throughput, and storage footprint

Excluded Scope:
- Real payment processing — transactions are simulated for demonstration purposes
- Complex shipping, logistics, and inventory management workflows
- User authentication via social login providers

- Mobile application development — the system targets desktop and tablet web browsers
- Custom model training or fine-tuning — this work evaluates pre-trained models exclusively

2.4.1 Known Limitations

This project has several limitations that should be acknowledged:

1. **Dataset size.** The evaluation uses 5,000 fashion product images from the Fashion Product Images dataset [7]. While sufficient for controlled comparative benchmarking, this is smaller than production catalogs that may contain millions of items. Results may not extrapolate directly to web-scale deployments.

2. **Hardware constraints.** All experiments were conducted on consumer-grade hardware with a mid-range GPU. A production deployment would likely benefit from dedicated inference servers with larger GPU memory and higher compute throughput. Reported latency and throughput figures should be interpreted relative to the experimental hardware profile.

3. **No user testing.** Due to scope constraints, the evaluation focuses exclusively on quantitative technical metrics — retrieval accuracy, inference latency, and throughput. Results were qualitatively reviewed by visual inspection of search output, but no formal user experience study was conducted. The relationship between measured accuracy metrics and subjective user satisfaction remains an open question for future investigation.

4. **Pre-trained models only.** All embedding models are used as published by their original authors, without fine-tuning on the evaluation dataset. Domain-specific fine-tuning might improve retrieval quality, particularly for models pre-trained on generic image corpora rather than fashion-specific data, but this was beyond the scope of the present work.

These limitations define starting points for future improvement and are revisited in the concluding chapter.

2.5 RESEARCH METHODOLOGY

This thesis adopts a Design Science Research (DSR) methodology [8], [9]. DSR is a problem-solving paradigm that produces and evaluates an IT artifact — in this case, a fashion e-commerce platform with integrated visual search — to address a defined problem domain. The methodology is structured around the iterative construction and assessment of a working system, with knowledge contributions derived from both the artifact itself and the empirical measurements obtained during its evaluation.

The project progressed through four phases. The **research and planning** phase involved surveying literature on visual search in fashion, exploring available pre-trained embedding models, evaluating database options for vector storage, and selecting the technology stack. The **design** phase formalized the system architecture — a modular .NET monolith with a Python AI sidecar communicating via HTTP — and structured the database schema to support both relational and vector data. The **implementation** phase built the three principal components: the .NET backend following vertical slice architecture, the Python embedding service, and the Vue.js storefront. The **test and evaluation** phase measured retrieval accuracy using standard information retrieval metrics, benchmarked inference latency and throughput, and conducted qualitative review of search output.

This methodology is appropriate for the project's goals because the primary contribution is not a theoretical advance in machine learning but an engineering demonstration: a working system that integrates academic model research into a conventional web stack, accompanied by empirical data on the performance trade-offs that practitioners face when making model and architecture decisions.

2.6 THESIS OUTLINE

This thesis is organized into seven chapters across three parts.

Part I: Introduction.

Chapter 1 — Introduction establishes the research context and motivation, defines the problem, states the objectives and research questions, delineates the scope and limitations, describes the methodology, and provides the present outline.

Part II: Thesis Content.

Chapter 2 — Background and Related Work provides the technical foundation necessary to understand the system. It covers vector embeddings and their mathematical role in similarity search, surveys convolutional and transformer-based neural network architectures for image feature extraction, examines vector database technologies with emphasis on pgvector, reviews prior academic and commercial work in fashion image retrieval, and concludes with a summary of the technology stack selected for this project.

Chapter 3 — Requirements Analysis translates the problem statement into concrete requirements for the system. It identifies functional and non-functional requirements organized by business domain, describes the user roles and their interactions with the platform, and presents use cases for the visual search workflow.

Chapter 4 — System Architecture and Design presents the architectural design of the platform. It describes the modular monolith structure and the bounded contexts of each business module, explains the sidecar pattern used to integrate the Python AI service with the .NET backend, details the database schema including the pgvector extension configuration, and documents the API design for the visual search pipeline.

Chapter 5 — Implementation describes the realization of the design in code. It covers the .NET backend implementation using vertical slice architecture, the Python embedding service built with FastAPI and PyTorch, and the Vue.js storefront application, including key patterns and conventions applied across the codebase.

Chapter 6 — Testing and Evaluation presents the empirical evaluation of the system. It reports the results of a systematic benchmark comparing retrieval accuracy across four embedding models using a three-fold cross-validation protocol on 5,000 fashion product images, measures inference latency and throughput on consumer-grade hardware, and analyzes the accuracy-speed trade-offs that inform model selection decisions.

Part III: Conclusion.

Chapter 7 — Conclusion and Future Work synthesizes the findings, evaluates the achievement of each research objective against the empirical results, discusses the significance and limitations of the work, and proposes directions for future research and system improvement.

PART 2: THESIS CONTENT

4 BACKGROUND AND RELATED WORK

4.1 E-COMMERCE PLATFORM ARCHITECTURES

Enterprise web applications have evolved through several architectural patterns, each shaped by the competing demands of development velocity, operational complexity, and runtime performance. The **monolithic** architecture packages all components — user interface, business logic, data access — into a single deployable unit. While simple to develop and test at small scale, monoliths tend to accumulate coupling between subsystems as the codebase grows, making independent feature development and incremental upgrades progressively more difficult. At the opposite pole, the **microservices** pattern decomposes an application into independently deployable services, each owning a discrete business capability. This enables teams to work in parallel and adopt different technology stacks per service, but it introduces substantial operational overhead: distributed data management, network latency, partial failure modes, and complex deployment pipelines. Between these extremes lies the **modular monolith**, an architecture that organises code into logically isolated business modules within a single process, sharing a common data store while enforcing strict compile-time boundaries that prevent direct cross-module references.

This thesis adopts the modular monolith pattern as the structural backbone of the ReSys.Shop platform. The choice is motivated by three practical concerns. First, a single-process architecture eliminates the need for service discovery, inter-service authentication, and distributed transaction orchestration — complexities that would be disproportionate for a system whose primary research contribution lies in its machine learning integration, not in its infrastructure. Second, a shared PostgreSQL database allows relational product data and vector embeddings to coexist within the same transactional context, an important property for maintaining consistency between catalog updates and search index changes. Third, the nine business modules (Catalog, Ordering, Payment, Inventory, Identity, Profile, Shipping, Location, and Dashboard) are separated by namespace convention with no direct cross-references, preserving the logical independence of bounded contexts while avoiding the operational cost of separate deployment units. The machine learning capability — image embedding generation and model inference — is isolated in a dedicated Python sidecar service, the one component that runs as a separate process due to its distinct technology stack and resource profile.

4.2 VISUAL SEARCH AND CONTENT-BASED IMAGE RETRIEVAL

At the heart of visual search is a simple idea: turning images into lists of numbers that a computer can compare. These lists are called **vector embeddings** or **feature vectors**.

4.2.1 Definition and Mathematical Representation

When we look at an image, we see colours, shapes, and patterns. Computers cannot "see" in the same way; they require numerical data to process information. A vector embedding is a way to represent the visual content of an image as a sequence of numbers.

For example, when an AI model processes an image of a red dress, it might output a 512-dimensional list like `[0.23, -0.15, 0.87, 0.42, ..., -0.31]`.

This list captures the "essence" of that image in a compressed form. Similar images will produce similar lists of numbers.

4.2.2 The Latent Space

When we talk about vectors, we can think of them as points in a high-dimensional space. For a 512-dimensional vector, imagine a space with 512 axes — instead of just the three axes we can visualise (x, y, z).

In this space:
- Similar images are located close together
- Different images are far apart

This space where embeddings live is called the **latent space**. The word "latent" means hidden, signifying that these dimensions do not correspond to obvious things like "redness" or "stripiness," but rather to abstract features the model learned during training.

4.2.3 Measuring Similarity

To find similar products, the system needs to measure how close two vectors are. This project uses **cosine similarity**, which measures the angle between two vectors:

$$\text{similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

Where:
- A and B are the two vectors being compared
- $A \cdot B$ is the dot product (multiply corresponding elements and sum)

- $\|A\|$ and $\|B\|$ are the lengths of each vector

The cosine similarity ranges from:
- **1.0** — vectors point in the same direction (very similar)
- **0.0** — vectors are perpendicular (unrelated)
- **−1.0** — vectors point in opposite directions (very different)

For fashion images, a cosine similarity above 0.7 typically indicates visual similarity that users would recognise.

4.2.4 Mathematical Similarity: From Visual Comparison to Cosine Distance

The power of embeddings is that complex visual comparisons become simple mathematics. Instead of trying to write rules like "match items with similar colours and patterns," the system:

1. Converts the query image to a vector
2. Compares that vector to all product vectors in the database
3. Returns products with the highest similarity scores

This approach is much more flexible than rule-based matching because the AI model learns which features are important from examples, rather than having those features hand-coded by a developer.

4.2.5 The CBIR Pipeline

The end-to-end flow from image upload to ranked results is a sequential pipeline connecting preprocessing, feature extraction, vector storage, and similarity search. Figure Figure 1 summarises this pipeline as implemented in the ReSys.Shop platform.

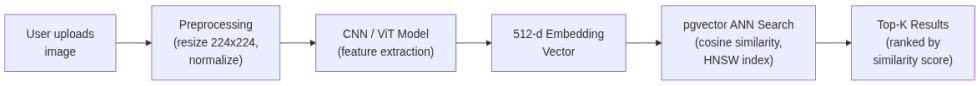

Figure 1.

Figure 1: High-level CBIR pipeline: from image upload through preprocessing, feature extraction, vector indexing, and similarity search to ranked results.

Vector embeddings are the mathematical foundation that enables visual search. The key question becomes: **how do we generate these embeddings?** This requires specialised neural network architectures that can extract meaningful features from images. The next section surveys the architectures relevant to this project.

4.3 DEEP LEARNING MODELS FOR FASHION IMAGE RETRIEVAL

Generating useful vector embeddings from fashion product images requires models that can extract features at multiple levels of abstraction — from low-level textures and edges to high-level concepts like silhouette, style category, and garment type. Over the past decade, three families of neural architectures have emerged as the dominant approaches to this task, progressing from convolutional feature hierarchies to attention-driven global context models and, most recently, to models that jointly understand both visual and textual descriptions.

4.3.1 Convolutional Neural Networks

Convolutional neural networks (CNNs) process images through a hierarchy of learned filters that detect increasingly abstract patterns [3]. Early layers respond to simple features — edges, colour gradients, and texture orientations — while deeper layers compose these primitives into complex structures such as fabric patterns, neckline shapes, and garment silhouettes. This hierarchical organisation mirrors aspects of the mammalian visual cortex and has proven remarkably effective for visual recognition tasks.

Two CNN architectures are particularly relevant to this project. **ResNet** (Residual Network), introduced by He et al. in 2016, addressed the vanishing gradient problem that had historically limited the depth of trainable networks. By adding skip connections that bypass one or more layers, ResNet enables gradients to flow directly through the network during backpropagation, making it feasible to train architectures with 50, 101, or even 152 layers. ResNet-50, the 50-layer variant, serves as a widely adopted feature extractor in the computer vision community, pre-trained on the ImageNet dataset of over one million labelled images across one thousand categories.

EfficientNet, proposed by Tan and Le in 2019, introduced a compound scaling method that uniformly scales network depth, width, and input resolution using a principled set of coefficients. This produces a family of models (EfficientNet-B0 through B7) that achieve better accuracy with fewer parameters than previous architectures. EfficientNet-B0, the smallest variant, produces 1,280-dimensional embeddings and is notable for its low computational footprint: it runs faster than most alternatives while retaining competitive feature quality, making it an attractive baseline for resource-constrained deployments [4].

4.3.2 Vision Transformers

While CNNs excel at capturing local patterns through their layered design, their reliance on small receptive fields — typically 3×3 or 5×5 pixel windows —

limits their ability to model relationships between distant image regions. Vision Transformers (ViTs) address this limitation by applying the self-attention mechanism originally developed for natural language processing to image data [10].

A ViT processes an image by dividing it into a grid of fixed-size patches (typically 16×16 pixels), flattening each patch into a token vector, and feeding the resulting sequence through a stack of transformer encoder layers. The self-attention mechanism within each layer computes pairwise relationships between all patches, allowing the model to capture long-range dependencies — a sleeve pattern that echoes a collar detail, a texture that repeats across a dress — that would require many successive CNN layers to approximate. This global receptive field is especially valuable for fashion retrieval, where the overall composition of a garment matters as much as its local details.

DINOv2, developed by Oquab et al., represents a particularly notable ViT variant for this project's use case [11]. Unlike earlier ViTs trained with human-annotated labels, DINOv2 uses self-supervised learning: it is trained to produce consistent representations across different augmentations of the same image, without requiring any category labels. This self-supervision objective encourages the model to discover semantically meaningful features directly from visual structure. DINOv2 ViT-S/14, the small variant with 14×14 patch size, produces 384-dimensional embeddings and has been shown to achieve strong performance on image retrieval tasks despite its compact size. The absence of any supervised classification head means that features are not biased toward the particular set of categories present in a labelled training dataset, making DINOv2 a strong candidate for open-ended visual search scenarios.

4.3.3 CLIP and Multimodal Models

CNNs and ViTs operate purely in the visual domain: they map an image to a point in an embedding space, but that space has no connection to human language. **CLIP** (Contrastive Language-Image Pre-training), introduced by Radford et al., bridges this gap through a dual-tower architecture [5]. One tower — typically a ViT — encodes images into vectors. A parallel text encoder tower processes natural language descriptions. Both are trained jointly on a dataset of 400 million (image, caption) pairs collected from the public web, using a contrastive objective that pulls matching image-text pairs together in a shared embedding space while pushing non-matching pairs apart.

The result is a **shared latent space** where an image of a dress and the phrase "red floral summer dress" map to nearby vectors, even though they originate from entirely different modalities. This property is the foundation of multimodal search: a user can query with text, with an image, or with a combination of both.

Fashion-CLIP extends this paradigm by fine-tuning CLIP on over 700,000 fashion images paired with domain-specific textual descriptions [6]. The fine-tuning process adjusts the model's weights to emphasise fashion-relevant attributes — garment categories, fabric textures, style descriptors, and occasion labels — while retaining the general visual understanding acquired during CLIP's broad pre-training. Fashion-CLIP uses the ViT-B/16 architecture (a ViT-Base with 16×16 patch size) inherited from CLIP, producing 512-dimensional embeddings. By narrowing the domain focus, Fashion-CLIP improves retrieval quality on fashion queries by approximately 10—20% relative to general CLIP, as demonstrated in the original paper and confirmed in the evaluation presented in Chapter 6 of this thesis.

The dual-tower design also makes Fashion-CLIP uniquely capable of multimodal queries. A user searching for "red floral summer dress" does not need to upload a reference image; the text encoder maps the query directly into the same embedding space as the catalog images. Similarly, a hybrid search combining an uploaded photo with a textual refinement — "like this, but in blue" — becomes possible by encoding both modalities and merging the resulting vectors.

4.3.4 Model Comparison

Table Table 1 summarises the models discussed in this section, spanning both convolutional and transformer-based architectures, general-purpose and domain-specific training objectives, and a range of embedding dimensionalities and inference latencies. The inference times are measured on the experimental hardware described in Chapter 6.

Table 1.

Table 1: Comparison of candidate embedding models for fashion product retrieval. Inference times are approximate and measured on a mid-range consumer GPU. Fashion-CLIP balances moderate dimensionality with domain-specific training and multimodal capability.

Model	Architecture	Embedding Dim	Training Method	Domain	Inference (ms)
ResNet-50	CNN	2048	Supervised (ImageNet)	General	15
EfficientNet-B0	CNN	1280	Supervised (ImageNet)	General	21
DINOv2 ViT-S/14	ViT	384	Self-supervised	General	80
CLIP ViT-B/16	ViT	512	Contrastive (image-text)	General	60
Fashion-CLIP ViT-B/16	ViT	512	Contrastive (fashion)	Fashion	68

4.4 VECTOR SEARCH AND DATABASES

Once images are converted to vector embeddings, those vectors must be stored and searched efficiently. A naive approach — comparing the query vector to every stored vector and sorting by distance — becomes impractical as the catalog grows. For a catalog of n products, each search requires n distance computations. At the scale of 10,000 items this is manageable on modern hardware; at 1,000,000 items it is several orders of magnitude too slow for interactive search.

4.4.1 Approximate Nearest Neighbour Search

The solution is **approximate nearest neighbour** (ANN) search. Rather than exhaustively computing the distance to every vector, ANN algorithms use index structures that organise vectors into a graph or tree, enabling the search to navigate directly toward the neighbourhood of likely matches. The key insight is that for product search, an approximate result with 97—99% recall of the true top matches is entirely acceptable, and the speed improvement can be several orders of magnitude.

4.4.2 HNSW: Hierarchical Navigable Small World

This project uses the **HNSW** algorithm, introduced by Malkov and Yashunin, which is among the most effective ANN approaches [12]. HNSW builds a multi-layer graph where each layer is a navigable small-world network. The topmost layer contains a sparse subset of nodes with long-range connections, functioning like the express lanes in a highway system. Lower layers are progressively denser, with shorter connections that enable fine-grained local search. A query begins at an entry point in the top layer, hops between nodes toward the query region in logarithmic time, then descends to the next layer to refine the result.

This layered structure reduces the search complexity from $O(n)$ for brute-force scan to approximately $O(\log n)$.

4.4.3 pgvector: Vector Search in PostgreSQL

pgvector is an open-source extension that adds vector storage and search operations to PostgreSQL [13]. It introduces a `VECTOR(d)` column type for d-dimensional vectors, provides cosine distance and Euclidean distance operators directly in SQL, and supports both HNSW and IVFFlat index types for accelerated similarity queries.

The decision to use pgvector rather than a specialised vector database — such as Pinecone, Milvus, or Weaviate — was driven by several practical considerations. First, pgvector stores vectors in the same database tables as the product metadata they describe, meaning that product updates and embedding updates occur within a single ACID transaction. This eliminates a class of bugs known as the **dual-database problem**, where a vector index can drift out of sync with its source of truth because the two stores have independent consistency guarantees. Second, pgvector requires no additional infrastructure: no separate service to deploy, monitor, and secure. For a system that already operates PostgreSQL for its relational data, adding pgvector is a matter of loading one extension. Third, pgvector queries use standard SQL, allowing developers to combine vector similarity search with relational filtering — find products visually similar to this image, but only from the dresses category and priced under one hundred dollars — in a single query with a single query plan. The trade-off is scale: pgvector is well-suited to catalogs in the tens of thousands to low millions of vectors, but may require migration to a dedicated vector database for web-scale deployments beyond that range.

4.4.4 Configuration and Distance Metric

Cosine distance is the primary comparison metric used in this project. For normalised embeddings, cosine distance and Euclidean distance are functionally equivalent — sorting by one produces the same ranking as sorting by the other — but cosine distance has the advantage of being bounded between 0 and 2 for normalised vectors, making similarity thresholds more interpretable. HNSW configuration parameters are left at their pgvector defaults (`m = 16` connections per node, `ef_construction = 64`), which provide a satisfactory balance of index build time and query accuracy for the catalog sizes evaluated in this work.

4.5 RELATED SYSTEMS

This section compares the ReSys.Shop platform with existing academic research and commercial products to understand the current state of fashion visual search and how this project contributes to the field.

4.5.1 Academic Research in Fashion Retrieval

Visual search for fashion has been an active research area for over a decade. Key developments include:

DeepFashion Dataset. The DeepFashion dataset, introduced by Liu et al., with over 800,000 fashion images, established benchmarks for fashion recognition and retrieval [14]. It provided attribute annotations (colour, pattern, category), landmark annotations (collar, sleeve, hemline positions), and pairs of matching in-shop and consumer photos. This dataset enabled much of the subsequent research in fashion AI.

Conversational Fashion Retrieval. More recent work has explored combining images with text feedback. FashionIQ introduced the task of modifying retrieval based on natural language — for example, "like this dress but shorter" [15]. This requires understanding both images and text modifications. While conceptually compelling, the interactive dialogue paradigm of conversational retrieval was beyond the scope of this project, which focuses on single-turn image and text queries.

Pre-trained Foundation Models. Recent trends favour using pre-trained models like CLIP rather than training from scratch. Fashion-CLIP demonstrated that domain-specific fine-tuning of CLIP improves fashion retrieval by 15—20% over general CLIP [6]. This project follows the same approach: using pre-trained Fashion-CLIP rather than training new models, and extending the evaluation to include additional architectures (ResNet, EfficientNet, DINOv2) for systematic comparison.

4.5.2 Commercial Visual Search Systems

Several companies have deployed visual search at scale:

Table 2.

Table 2: Comparison of commercial visual search products.

Product	Strengths	Limitations for This Project
Google Lens	Massive scale, general purpose	Closed ecosystem, not customisable
Pinterest Lens	600M+ monthly searches, style-aware	Proprietary, requires Pinterest integration
ASOS Style Match	Fashion-specific accuracy	Only for ASOS catalog
ViSenze	API available, good accuracy	Paid service, recurring costs

These products are impressive but share common limitations for smaller projects: they are proprietary and cannot be studied or modified; API access incurs costs per query; and relying on an external service creates vendor lock-in. This project demonstrates that similar functionality can be achieved with open-source tools, providing a reference implementation and cost-effective solution for smaller applications.

4.5.3 Technical Positioning and Contribution

This project distinguishes itself from existing literature by addressing the **engineering gap** between theoretical AI models and production-grade software. While typical research focuses on optimising metric scores (e.g., mAP), this thesis contributes a reference architecture for **operationalising** those models.

4.5.3.1 1. Polyglot Vertical Slice Architecture

Most open-source implementations force a choice between a monolithic Python web stack (Django/Flask) or a complex microservices mesh. This project introduces a **Distributed Vertical Slice** pattern that:

- Leverages **.NET 10** for strict type safety, high-performance concurrency, and domain logic integrity in the transactional core.
- Isolates **Python 3.12** solely for tensor computations (PyTorch), connected via a resilient HTTP bridge.
- **Differentiation:** This provides the best of both worlds — enterprise-grade backend reliability with access to the bleeding-edge AI ecosystem, without the operational overhead of a full microservices deployment.

4.5.3.2 2. Vector-Native Data Consistency

A common pitfall in visual search is the dual-database problem, where vector data (stored in a Chroma/Pinecone instance) drifts from the relational source of truth (SQL).

- **Contribution:** This implementation utilises **pgvector** to enforce ACID transactions across both relational entities and vector embeddings.

- **Impact:** Product updates and index re-calculations occur in the same atomic transaction scope, eliminating the class of "stale index" bugs common in distributed systems.

4.5.3.3 3. Feature Parity on Commodity Hardware

Commercial solutions such as Google Lens rely on massive cloud TPU clusters. This project demonstrates that **Fashion-CLIP** (ViT-B/16) can be effectively served on commodity hardware (mid-range GPU or standard CPU) with sub-100ms latency.

- **Result:** This lowers the barrier to entry for small and medium-sized e-commerce platforms to adopt generative AI and semantic search features without recurring cloud API costs.

4.5.3.4 4. Applied Evaluation of Foundation Models

Moving beyond the "leaderboard" mentality, this thesis compares **ResNet**, **EfficientNet**, **DINOv2**, and **Fashion-CLIP** specifically within the constraints of a real-time web application.

- **Key Finding:** We demonstrate that while DINOv2 offers superior raw geometric matching, its larger computational cost makes it less viable for CPU-bound environments than the optimised Fashion-CLIP, providing a pragmatic guide for model selection in resource-constrained deployments.

4.6 TECHNOLOGY STACK

The ReSys.Shop platform is built on a polyglot stack spanning three programming languages — C#, TypeScript, and Python — orchestrated through a unified containerised development environment. Table Table 3 summarises the principal technologies and their roles.

Table 3.

Table 3: Technology stack of the ReSys.Shop platform, organised by architectural layer.

Layer	Technology	Purpose
Frontend	Vue 3 + TypeScript + Vite	Customer storefront and admin panel. Vue 3 provides a reactive component model; TypeScript adds type safety to frontend code; Vite delivers fast development builds. State management is handled by Pinia stores.
Backend API	.NET 10 + Carter + MediatR	REST endpoints delivered via Carter minimal APIs. MediatR implements CQRS (Command Query Responsibility Segregation), cleanly separating read and write operations and enforcing the modular boundaries between business domains.
Database	PostgreSQL + pgvector	Relational data and vector embeddings coexist within a single ACID database. The pgvector extension provides HNSW-indexed vector similarity search using standard SQL, eliminating the need for a separate vector store.
Caching	Redis + HybridCache	Multi-tier caching with in-memory L1 (fast, local) and Redis L2 (shared, durable). Also serves as the backing store for Hangfire job queues and session state.
ML Sidecar	Python + FastAPI + PyTorch	Dedicated service for image embedding generation. FastAPI provides async HTTP endpoints with built-in OpenAPI documentation. PyTorch executes model inference and supports GPU acceleration when available.
Orchestration	.NET Aspire	Service discovery, container lifecycle management, and local development environment. Aspire coordinates the startup and interconnection of the .NET API, Python ML sidecar, PostgreSQL, and Redis containers.
Background Jobs	Hangfire	Persistent job processing for cart expiry (auto-clean after 7 days of inactivity), embedding generation queue (decoupling image upload from inference), and periodic maintenance tasks.
Authentication	JWT + ASP.NET Identity	Short-lived access tokens (15-minute lifetime) with refresh token rotation and reuse detection. Role-based and permission-based authorisation segregates admin functions from customer-facing endpoints.

Each technology was selected to satisfy a specific architectural requirement. Vue 3 and Vite provide a modern frontend development experience with fast iteration cycles appropriate for the scope of a single-developer thesis project. .NET 10 offers the strong type system, high-throughput web server, and mature ORM (Entity Framework Core) that underpin the transactional e-commerce backend. PostgreSQL was a natural choice for the data layer because it hosts both the relational schema and, via pgvector, the vector index — consolidating data management into a single well-understood database. Redis adds the caching tier that keeps repeated queries fast and stores transient state. The Python sidecar exists specifically to host PyTorch, the de facto standard framework for loading and running pre-trained deep learning models, isolated from the main application process so that a failure or resource spike in inference does not affect the availability of the e-commerce API. .NET Aspire unifies these components into a reproducible development environment using container-based orchestration. Hangfire ensures that background operations survive application restarts by persisting job state in Redis. Finally, JWT-based authentication with refresh token rotation follows current best practices for securing browser-based single-page applications.

5 REQUIREMENTS ANALYSIS

This chapter defines the scope of the ReSys.Shop platform by identifying its actors, functional and non-functional requirements, key use cases, and the classification of features into research contributions and supporting infrastructure. The analysis establishes what the system must do before proceeding to its architectural design and implementation.

5.1 SYSTEM ACTORS

The platform serves three categories of actors, each with a distinct role, set of permissions, and interaction surface. Table Table 4 summarises these actors and their primary responsibilities.

Table 4.

Table 4: System actors and their roles within the ReSys.Shop platform.

Actor	Role Description	Interaction Surface
Customer (Guest + Authenticated)	Browses the product catalog, performs keyword and visual searches, manages a shopping cart, completes multi-step checkout, and tracks order history. Guest users can browse and add items to a cart; authenticated users access profile management, wishlists, and personalised features.	Vue 3 Storefront SPA\ (Web browser)
Administrator	Manages the full product lifecycle: creating and updating products with fashion-specific metadata, uploading and organising product images, defining taxonomies, monitoring inventory levels, processing order fulfilment, and managing user accounts and permissions.	Vue 3 Admin SPA\ (Web browser)
System (Background Services)	Automated background processes that maintain data consistency and system performance: generating and indexing vector embeddings for newly uploaded images, expiring abandoned carts after a configurable time window, reserving and releasing inventory during checkout, and performing periodic index maintenance.	Internal services\ (No direct UI)

The Customer and Administrator actors represent human users interacting through browser-based single-page applications. The System actor represents background processes that operate without direct human interaction, executing scheduled and event-driven tasks through Hangfire job workers within the .NET application process. The three actors together define the complete set of interactions supported by the platform.

5.2 FUNCTIONAL REQUIREMENTS

The functional capabilities of ReSys.Shop are organised around eight business modules, each responsible for a coherent subset of domain logic. This section describes each module in narrative prose; a summary table at the end of the section consolidates responsibilities and research classification.

5.2.1 Catalog Module

The Catalog module manages the product lifecycle: creating products with fashion-specific metadata — including style code, season, material, department, and gender target — defining sellable variants with SKUs, barcodes, and independent pricing, uploading product images with automatic thumbnail generation, and organising products through hierarchical taxonomies that allow browsing by category (e.g., Clothing → Dresses → Evening Dresses). It also hosts the Content-Based Image Retrieval (CBIR) infrastructure: newly uploaded variant images are sent to the Python machine learning sidecar for vectorisation, and the resulting embeddings are stored in PostgreSQL using the pgvector extension for similarity search [13]. The catalog supports configurable embedding models, allowing the system to switch between Fashion-CLIP, ResNet-50, and other architectures without application changes — a capability that enables the systematic benchmark evaluation presented in Chapter 6.

5.2.2 Ordering Module

The Ordering module handles the customer purchase workflow from cart to completed order. Both guest and authenticated users can add products to a cart, which automatically expires after seven days of inactivity to prevent indefinite stock reservation. Checkout proceeds through a forward-only state machine: the customer selects a shipping address, chooses a delivery method, provides payment details, reviews the order summary, and confirms. Once confirmed, the system creates an order record, reserves inventory quantities for each line item, processes the payment intent, and clears the cart. Orders track item totals, price adjustments, shipment costs, and payment state independently, enabling partial fulfilment scenarios. Cancellation is available at any pre-confirmation stage without penalty.

5.2.3 Payment Module

The Payment module manages the lifecycle of payment intents — the data structure representing a customer's intent to pay — including creation, capture, refund, and void operations. It supports two gateway providers: the Stripe gateway for production, which validates incoming webhooks using signature verification to prevent spoofed payment confirmations, and a Bogus gateway

for development and testing, which simulates the payment lifecycle by automatically transitioning through states without external calls. Payment intents follow their own state machine — Pending, RequiresAction, Processing, and Succeeded or Canceled — and the system maintains its own copy of the payment state in parallel with the gateway's state, enabling offline operations and consistent behaviour across both gateway implementations.

5.2.4 Inventory Module

The Inventory module tracks physical stock quantities across warehouse locations, manages temporary reservations during active checkouts to prevent overselling, records stock movements for audit trails, and handles inter-warehouse transfers. Each stock item is associated with a specific product variant and a warehouse location, with quantities maintained as both on-hand (physical count) and reserved (held for active checkouts) values. The reservation mechanism ensures that a variant added to a cart remains visible to other customers as having limited availability but cannot be sold twice.

5.2.5 Identity Module

The Identity module provides JWT-based authentication with short-lived access tokens (fifteen-minute lifetime) and refresh token rotation with reuse detection: each refresh token is single-use, and presenting a previously consumed token triggers revocation of all tokens for that user to contain potential compromise. Guest sessions enable anonymous cart usage through cookie-based identifiers that persist across page navigations without requiring account creation. Role-based and permission-based authorisation segregates admin functions from customer-facing endpoints, with permission claims following a `domain:category:action` format that allows fine-grained access control.

5.2.6 Supporting Modules

Three additional modules provide complementary infrastructure. The **Profile** module manages user addresses, wishlists, and notification preferences, linking customer identity to personalisation features. The **Shipping** module configures delivery methods — standard, express, and local pickup — and calculates shipping rates by geographic zone. The **Location** module provides country and state reference data with ISO codes, shared across Shipping (for zone configuration) and Profile (for address validation).

5.2.7 Summary

Table Table 5 consolidates the eight business modules with their key responsibilities and research classification relative to this thesis.

Table 5.

Table 5: Summary of business modules, their key responsibilities, and classification as Core Research or Supporting Infrastructure. Only Catalog is classified as Core Research because it hosts the CBIR capability that is the primary subject of evaluation.

Module	**Key Responsibilities**	**Research Classification**
Catalog	Product and variant lifecycle management; fashion-specific metadata (style code, season, material, department); hierarchical taxonomies; image upload and management; CBIR infrastructure — embedding generation pipeline and pgvector vector search.	Core Research
Ordering	Shopping cart with auto-expiry; forward-only checkout state machine (Address → Delivery → Payment → Confirm → Complete); order lifecycle tracking; cancellation and partial fulfilment support.	Supporting
Payment	Payment intent lifecycle (create, capture, refund, void); Stripe gateway with webhook signature validation; Bogus test gateway; parallel state tracking for offline operations.	Supporting
Inventory	Per-warehouse stock tracking; checkout-time quantity reservation to prevent overselling; auditable stock movements; inter-warehouse transfers.	Supporting
Identity	JWT authentication with 15-minute access tokens; refresh token rotation and reuse detection; guest session support; role-based and permission-based authorisation.	Supporting
Profile	User addresses (shipping and billing); wishlist management; notification preferences.	Supporting
Shipping	Delivery method configuration; zone-based rate calculation.	Supporting
Location	Country and state reference data with ISO codes.	Supporting

The functional scope of ReSys.Shop extends far beyond the visual search capability at its core. The supporting modules — Ordering, Payment, Inventory, and Identity — provide a realistic e-commerce context in which the research contribution can be meaningfully evaluated. Without a functioning checkout flow, for example, the value of visual search could not be measured through downstream conversion events. Without inventory awareness, search results could include out-of-stock items, undermining the realism of the evaluation.

5.3 NON-FUNCTIONAL REQUIREMENTS

Beyond feature completeness, the system must satisfy quantitative and qualitative constraints that determine its fitness for production use. Table Table 6 summarises the non-functional requirements across five quality dimensions.

Table 6.

Table 6: Non-functional requirements with concrete targets and design rationale.

Quality	Target	Rationale
Performance	CBIR end-to-end search latency under 1 second (image upload through embedding generation, vector database query, and result assembly). Non-search API endpoints respond within 200 milliseconds under normal load. Asynchronous I/O handles concurrent requests without blocking threads.	Real-time visual search requires sub-second response to maintain user engagement. Studies show that search latency above one second measurably increases abandonment rates in e-commerce contexts [16].
Security	JWT access tokens expire after 15 minutes; refresh tokens follow single-use rotation with reuse-detection invalidation. Role-based authorisation enforced per endpoint. Rate limiting on authentication endpoints (five requests per minute for login, three per hour for registration). Security response headers (CSP, HSTS, X-Frame-Options, X-Content-Type-Options) on all HTTP responses. File upload validation: magic-byte verification, extension allowlist, and 10 megabyte size limit.	Browser-based single-page applications are exposed to the open internet and must defend against common web threats. Short-lived tokens and refresh rotation limit the window of token compromise. File upload validation prevents malicious payloads from entering the embedding pipeline.
Modularity	Eight business modules in a single .NET assembly, separated by namespace convention with no direct cross-references. All inter-module communication occurs through MediatR in-process message dispatch. Each module independently testable without loading its neighbours.	The modular monolith pattern preserves the logical separation of microservices while avoiding distributed-system complexity. MediatR dispatch provides a clean integration point that can be replaced with an external message broker if modules are later extracted into separate services.
Observability	OpenTelemetry distributed tracing across .NET API and Python ML sidecar, with trace context propagation through HTTP headers. Structured logging with correlation identifiers on every log entry. Health check endpoints for each service, consumed by .NET Aspire for container orchestration and restart decisions.	In a polyglot architecture spanning C# and Python, end-to-end request tracing is essential for diagnosing latency bottlenecks and error propagation. Correlation identifiers enable a single request to be followed across service boundaries in log aggregators.
Reliability	Background jobs (cart expiry, embedding generation retries, index maintenance) persist in Redis-backed Hangfire storage, surviving application restarts without data loss. Payment webhooks include idempotency keys that prevent duplicate processing on retry. Cart expiry triggers after fifteen minutes of inactivity, releasing reserved inventory automatically.	Many e-commerce operations are inherently long-running or time-delayed — cart expiry, payment confirmation, and index maintenance. A durable job queue ensures these operations complete reliably, even across process crashes or scheduled restarts.

These non-functional requirements shaped architectural decisions throughout the system. The one-second CBIR latency target influenced the choice of a synchronous embedding pipeline rather than a queued approach; the modularity requirement led to the MediatR-based in-process dispatch model; and the reliability constraint motivated the choice of Hangfire with Redis-backed persistence for background jobs. Each target is revisited in the evaluation chapter, where the benchmark results confirm whether the implemented system meets these stated requirements.

5.4 USE CASES

This section presents three use cases that represent the system's core functional scenarios: visual search (the primary research capability), checkout (the primary e-commerce transaction), and model benchmark evaluation (the research methodology for Chapter 6). Each use case is described in a compact tabular format comprising the actor, preconditions, main flow as numbered sequential steps, and postconditions. Figure Figure 2 provides a visual summary of actor-system interactions.

5.4.1 Use Case 1: Visual Search (CBIR)

Table 7.

Table 7: UC-1: Visual Search (CBIR) — the primary research use case.

Actor	Customer (Guest or Authenticated)
Precondition	The Python ML sidecar service is running and has loaded the configured embedding model into memory. The product catalog contains at least one variant with a stored embedding vector for the active model.
Main Flow	1. Customer uploads a reference image (JPEG, PNG, or WebP; maximum ten megabytes) via the storefront visual search interface. 2. The Vue frontend sends the image as a multipart form data request to the .NET API endpoint. 3. The API validates the image — magic-byte verification, extension check, size limit — then forwards the raw image bytes to the Python ML sidecar. 4. The ML sidecar preprocesses the image (resize, normalise) and executes a forward pass through the configured embedding model, producing a floating-point vector. 5. The API queries PostgreSQL pgvector using cosine similarity against all stored variant embeddings filtered by the active model name, retrieving the top 20 most similar results. 6. The API joins variant data with product metadata, computes similarity scores, filters by a minimum similarity threshold (default 0.7), and returns the ordered results as JSON. 7. The Vue storefront renders the results as a grid of product thumbnails with similarity scores and prices.
Postcondition	A ranked list of visually similar products is displayed to the customer, ordered by decreasing cosine similarity. Each result includes the product thumbnail, name, price, and similarity score.

5.4.2 Use Case 2: Multi-Step Checkout

Table 8.

Table 8: UC-2: Multi-Step Checkout — the primary e-commerce transaction use case.

Actor	Customer (Guest or Authenticated)
Precondition	The customer's cart contains at least one valid, in-stock item. The customer has a shipping address on file or is prepared to enter one.
Main Flow	1. Customer clicks "Proceed to Checkout" from the cart page. 2. System presents the checkout interface, showing the current cart contents with item totals. 3. Customer selects or enters a shipping address. 4. Customer selects a delivery method from available shipping options. 5. Customer selects a payment method and provides payment details. 6. Customer reviews the order summary (items, shipping cost, tax, total) and clicks "Place Order". 7. System begins an atomic transaction: creates the order record, reserves inventory quantities for each line item, processes the payment through the configured gateway, and clears the cart. 8. System displays the order confirmation page with the order number and summary.
Postcondition	An order record is created with status "Placed". Inventory quantities for each ordered variant are reserved. A payment intent is linked to the order. The customer's cart is emptied. A confirmation is displayed with the order reference number.

5.4.3 Use Case 3: Model Benchmark Evaluation

Table 9.

Table 9: UC-3: Model Benchmark Evaluation — the research methodology use case.

Actor	Researcher / System
Precondition	The benchmark dataset is available on disk, consisting of query images and catalog images organised into human-labelled similarity groups. The Python ML sidecar is running. All candidate embedding model weights are downloaded and accessible.
Main Flow	1. Researcher selects a model from the candidate set (e.g., Fashion-CLIP, ResNet-50, DINOv2-S) and configures the ML sidecar via environment variable. 2. System generates embedding vectors for all query images and all catalog images using the selected model. 3. For each query image, the system executes a top-K (K = 20) similarity search against the catalog embeddings. 4. System computes retrieval metrics: Mean Average Precision (mAP), Precision at K, and Recall at K, using the human-labelled groups as ground truth. 5. System records operational metrics: average inference time per image, throughput (images per second), disk storage for the embedding index, and RAM consumption. 6. Steps 1—5 are repeated for each of the 11 candidate models. 7. System aggregates all results into comparison tables, ranking models by retrieval accuracy and operational efficiency.
Postcondition	A complete benchmark report is produced containing accuracy metrics (mAP, P@20, R@20) and efficiency metrics (latency, throughput, storage, RAM) for every evaluated model. The report identifies the optimal model for each deployment scenario (GPU production, CPU-only, maximum accuracy, resource-constrained).

Figure Figure 2 positions these three use cases alongside the broader system functionality within a single visual summary.

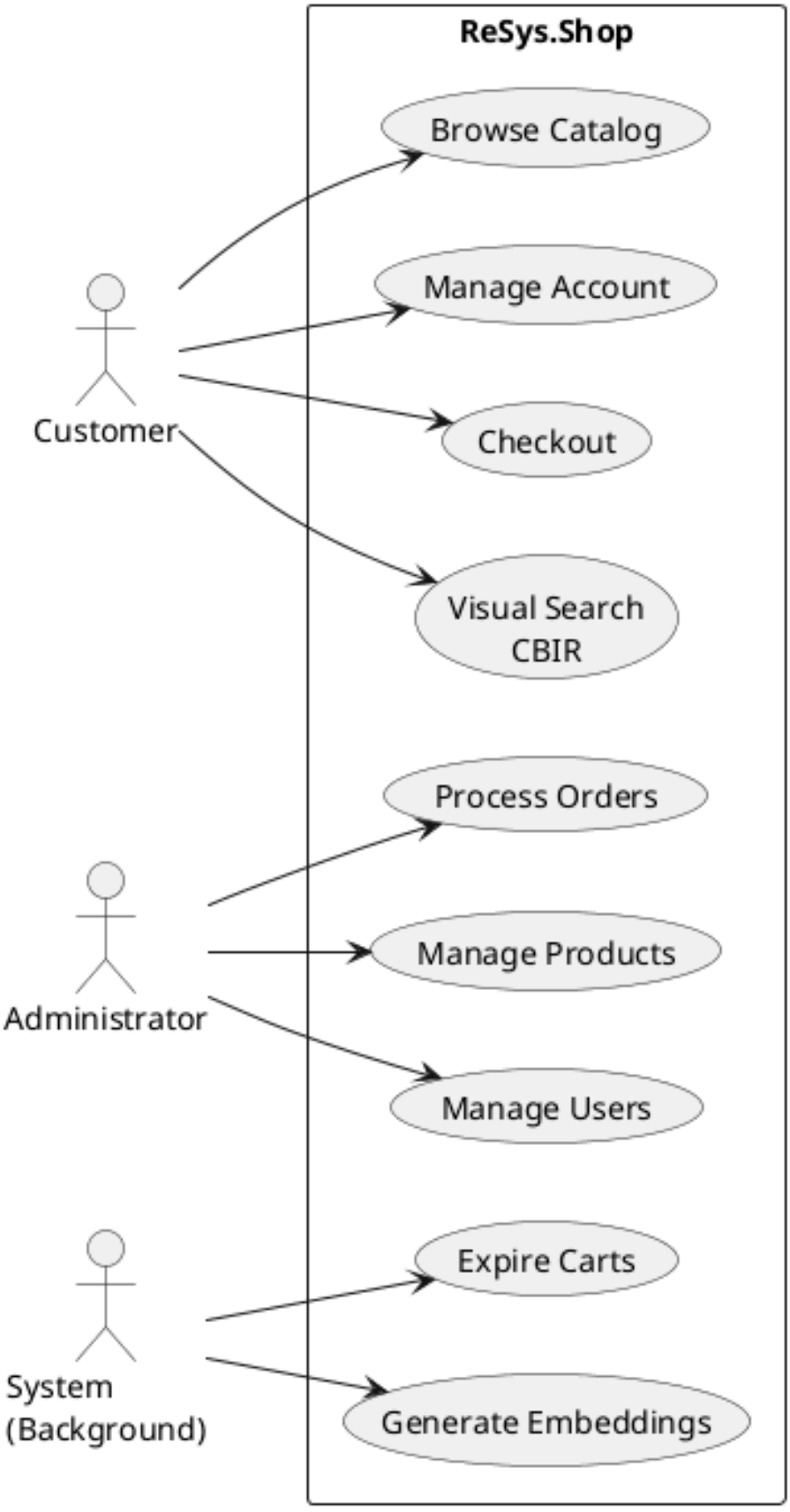
ReSys.Shop
Browse Catalog
Manage Account
Checkout
Visual Search
CBIR
Process Orders
Manage Products
Manage Users
Expire Carts
Generate Embeddings
Customer
Administrator
System
(Background)

Figure 2.

Figure 2: System use case diagram showing the three actors — Customer, Administrator, and System background services — and their primary interactions with the ReSys.Shop platform.

The three use cases serve distinct purposes within the thesis. The visual search use case defines the functional behaviour of the system's primary research capability; the checkout use case establishes the realistic e-commerce context in which search success can be measured through downstream conversion events; and the benchmark use case defines the systematic methodology used in Chapter 6 to evaluate and compare embedding models. The breadth of the system — nine background actors and use cases in the diagram, encompassing catalog browsing, account management, product administration, and order processing — reflects the full operational scope of the platform, while the three detailed use cases focus on the scenarios most relevant to the research questions.

5.5 FEATURE CLASSIFICATION

Not all features of ReSys.Shop carry equal research significance. Seven feature areas are classified in Table Table 10 as either **Core Research** (directly contributing to the thesis's academic objectives) or **Supporting Infrastructure** (providing the realistic e-commerce context in which the research is conducted and evaluated). This distinction is important for two reasons: it clarifies the scope of the thesis's original contribution, and it explains why certain features — shipping calculation, user management, country reference data — exist in the platform but are not discussed in depth in subsequent chapters.

Table 10.

Table 10: Classification of feature areas into Core Research and Supporting Infrastructure. Core Research features represent the thesis's original contributions; Supporting Infrastructure features provide the realistic e-commerce context necessary for meaningful evaluation.

Feature Area	Classification	Rationale
Visual Search (CBIR)	Core Research	Primary contribution: integrated multi-model CBIR with pluggable architecture enabling image-based product search across multiple embedding models (CNN, ViT, CLIP-based) within a production-style e-commerce platform.
ML Embedding Pipeline	Core Research	Critical infrastructure: automated ingestion of product images, generation of vector embeddings via the Python sidecar, storage in pgvector, and HNSW indexing — the operational backbone of the visual search capability.
Model Benchmark System	Core Research	Secondary contribution: systematic protocol for comparing retrieval accuracy and operational efficiency across 11 embedding models, providing a practical guide for model selection in resource-constrained deployments.
Product Catalog	Supporting Infrastructure	Required context: provides the structured dataset of fashion products — with variants, images, taxonomies, and metadata — that serves as the search target for CBIR evaluation.
Order System	Supporting Infrastructure	Metric validation: provides conversion events (add-to-cart, checkout completion) that serve as proxy indicators of search success, enabling the evaluation of visual search within a realistic shopping workflow.
Inventory	Supporting Infrastructure	Realism constraint: ensures that search results reflect actual product availability, preventing the unrealistic scenario where visually similar but out-of-stock items appear in search results.
Authentication	Supporting Infrastructure	Security baseline: protects administrative functions and user-specific data, enabling the application to operate in a representative security posture without which the system would be

The classification makes explicit what the thesis does and does not claim as contribution. The CBIR pipeline — encompassing embedding generation, vector storage, and similarity search — is the core research artefact. The e-commerce modules (Catalog, Ordering, Inventory, Payment, Identity) are supporting infrastructure, built to provide a realistic context that validates the visual search results in a production-like environment. This separation is maintained throughout the thesis: Chapters 4 and 5 dedicate detailed treatment to the research features, while the supporting infrastructure is described only to the extent necessary to understand the system's design.

6 SYSTEM ARCHITECTURE & DESIGN

This chapter presents the architectural design of ReSys.Shop, progressing from a high-level system overview through domain modelling, to the structural, data, API, and security layers. The design follows a service-oriented approach with three independently deployable services — a Vue 3 frontend, a .NET 10 modular monolith backend, and a Python machine learning sidecar — each responsible for a distinct technological concern. The chapter is organised into six sections, each accompanied by architectural diagrams that provide visual representations of the system's structure, behaviour, and deployment topology.

6.1 SYSTEM OVERVIEW

ReSys.Shop is built as a service-oriented system with three distinct services. The frontend is implemented in Vue 3 and TypeScript using the Vite build tool. The backend is a .NET 10 modular monolith using ASP.NET Core for HTTP handling, Entity Framework Core for data access, and Carter for minimal API endpoint registration. The machine learning service is a Python FastAPI application running PyTorch models that generates vector embeddings from product images for visual similarity search.

Table Table 11 summarises the three services, their technology stacks, and their primary responsibilities within the platform.

Table 11.

Table 11: System services and their technology stacks. Each service is independently deployable and communicates through well-defined HTTP contracts.

Service	Technology Stack	Responsibilities
Vue Frontend	Vue 3 + TypeScript + Vite	• Customer storefront (Nuxt UI) • Administrator dashboard (PrimeVue) • Pinia state management • Image upload and visual search UI
.NET Backend	.NET 10 + ASP.NET Core + Carter + EF Core	• REST API endpoints via Carter minimal APIs • Business logic via MediatR CQRS pattern • PostgreSQL persistence with pgvector vector search • JWT authentication and RBAC authorisation
Python ML	Python 3.12 + FastAPI + PyTorch	• Fashion-CLIP and other embedding model inference • Vector embedding generation from product images • Multi-model support with lazy-loading strategy

The backend is internally organised into eight bounded contexts following the principles of Domain-Driven Design. Each context owns a distinct area of business logic and communicates with other contexts exclusively through MediatR in-process message dispatch — there are no direct namespace references between business modules. Table Table 12 lists each context, its aggregate root, and a representative sample of its domain entities.

Table 12.

Table 12: Bounded contexts with aggregate roots and key domain entities. Each context owns its database schema and communicates with other contexts through MediatR dispatch only.

Bounded Context	Aggregate Root	Key Domain Entities
Catalog	Product	Variant, VariantImage, OptionType, OptionValue, Classification, Taxonomy, Taxon
Ordering	Order	LineItem, Adjustment, Shipment
Payment	PaymentIntent	PaymentCapture
Inventory	StockItem	StockLocation, StockMovement, StockReservation
Identity	User	Role, UserRole, RefreshToken, UserLogin
Profile	UserProfile	Address, Wishlist
Shipping	ShippingMethod	ShippingRate, ShippingZone
Location	Country	State

The separation of concerns across these eight contexts enables independent evolution of each business domain while the modular monolith deployment model avoids the operational complexity of distributed microservices. The following section details the domain-driven design principles that govern these contexts.

6.2 DOMAIN-DRIVEN DESIGN

The ReSys.Shop platform applies Domain-Driven Design (DDD) to structure its business logic around eight bounded contexts, each with well-defined aggregate roots, domain entities, and invariants. This section presents the context map, the aggregate design with invariants, the ubiquitous language glossary, and the state machines that govern the checkout and payment lifecycles.

6.2.1 Bounded Context Map

The eight bounded contexts partition the e-commerce domain along business capability boundaries. Each context owns its data, its domain logic, and its vocabulary — terms that are well-defined within a context may carry different meaning in another. For example, a Variant in the Catalog context is a sellable unit with a SKU and pricing; a LineItem in the Ordering context references that same variant but from the perspective of purchase fulfilment.

The integration between contexts follows the **Conformist** pattern: all contexts conform to a shared technical kernel defined in the Shared layer, which provides the `Result<T>` return type, the `ICommand` and `IQuery` marker interfaces, and the `Entity` base class with audit and versioning columns. Communication occurs exclusively through MediatR `ISender` — a context dispatches a query

or publishes a notification, and other contexts react without ever importing one another's namespace. This in-process dispatch model eliminates the network latency of inter-service messaging while preserving the logical isolation of the bounded contexts.

Figure Figure 3 depicts the eight contexts and the **Published Language** — the shared identifiers and value types — that flow between them.

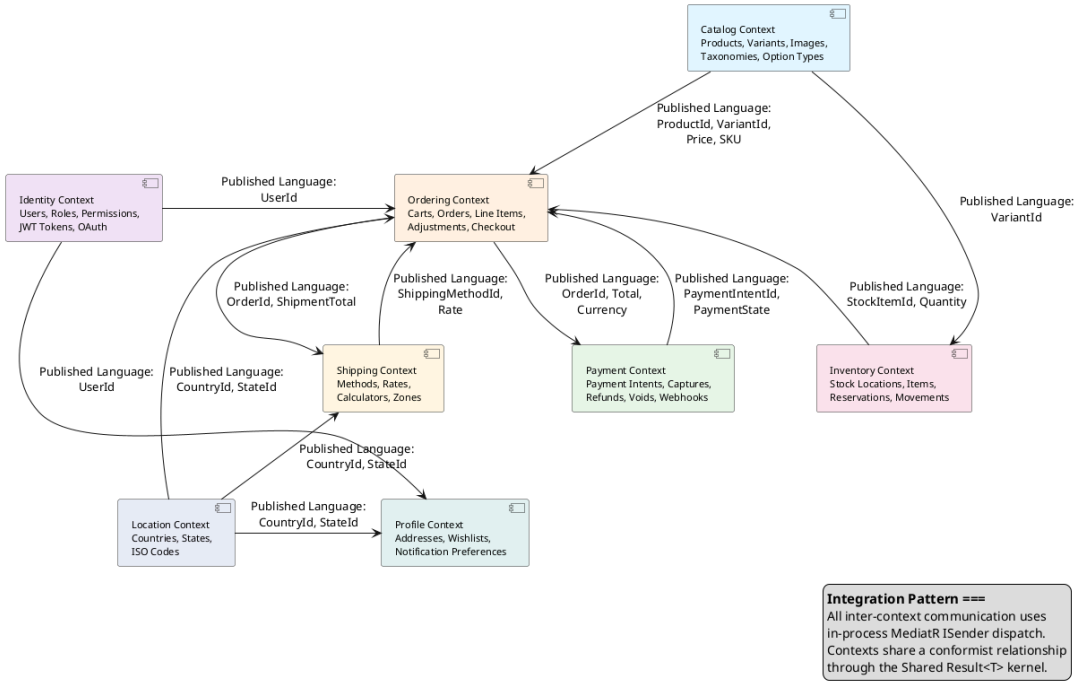

Figure 3.

Figure 3: Bounded Context Map showing the eight business contexts and the Published Language identifiers exchanged between them. All integration uses in-process MediatR dispatch; no context directly references another context's namespace.

Table Table 13 details each context's business responsibility and the integration data it exposes to the system.

Table 13.

Table 13: Bounded context responsibilities and Published Language identifiers. The Published Language column lists the value types that other contexts may reference by identifier only — never by importing the source context's namespace.

Context	Business Responsibility	Published Language
Catalog	Manages the product lifecycle: creating products with fashion-specific metadata (style code, season, material, department, gender target), defining sellable variants with SKUs and independent pricing, uploading images with automatic embedding generation, and organising products through hierarchical taxonomies.	ProductId, VariantId, Sku, Price, Slug
Ordering	Orchestrates the customer purchase workflow from cart to completed order. Manages cart with seven-day auto-expiry, forward-only checkout state machine, line items with price snapshots, adjustments, and cancellation at any pre-confirmation stage.	OrderId, OrderNumber, Total, Currency, CheckoutState
Payment	Manages payment intent lifecycle — creation, capture, refund, void — across two gateway implementations. Maintains parallel payment state independent of the gateway for offline operations and consistent behaviour across providers.	PaymentIntentId, PaymentState, Amount
Inventory	Tracks physical stock quantities per warehouse, manages temporary reservations during active checkouts to prevent overselling, records auditable stock movements through an append-only ledger, and handles inter-warehouse transfers.	StockItemId, QuantityOnHand, QuantityReserved
Identity	Provides JWT-based authentication with refresh token rotation and reuse detection, role-based and permission-based authorisation with `domain:category:action` claim format, and guest session management for anonymous browsing.	UserId, Email, PermissionClaim
Profile	Manages user addresses for shipping and billing, wishlists for product bookmarking, and notification preferences controlling email and SMS communication channels.	ProfileId, AddressId
Shipping	Configures delivery methods — standard, express, local pickup — and calculates shipping rates by geographic zone using weight- and distance-based calculators.	ShippingMethodId, Rate
Location	Provides country and state reference data with ISO 3166 codes. This context is read-only reference data shared across Shipping (zone configuration), Profile (address validation), and Ordering (checkout address selection).	CountryId, StateId, IsoCode

6.2.2 Aggregates and Invariants

An aggregate is a cluster of domain objects treated as a single consistency boundary. Each aggregate has a root entity through which all modifications must pass. The root enforces invariants — business rules that must hold true at all times within the aggregate boundary. ReSys.Shop takes a pragmatic approach to DDD: it defines aggregate roots and their invariants explicitly but does not require formal value-object base classes or a dedicated domain-event infrastructure for every operation.

The four most architecturally significant aggregates are described below.

Product (Catalog aggregate root). The Product aggregate encapsulates a product family and all its variants, images, option configurations, and taxonomy classifications. A product may have one or more variants; exactly one is designated as the master variant displayed on listing pages. The aggregate enforces the following invariants: every product must have a unique slug for SEO-friendly URL generation; a product that declares options (such as size or colour) must have at least one option type defined; and the master variant must exist among the product's own variants. Variant images contain the `embedding` column — a 512-dimensional float vector generated by the ML sidecar — enabling cosine similarity search against the entire image corpus.

Order (Ordering aggregate root). The Order aggregate manages the checkout lifecycle from a nascent cart through to a completed purchase. It aggregates line items, each capturing a price snapshot of the variant at the time of purchase, and optional adjustments for discounts or promotions. The aggregate enforces the invariant that `Total = ItemTotal + AdjustmentTotal + ShipmentTotal`, maintaining financial consistency across all modifications. The checkout state progresses forward only — the address, delivery, payment, and confirmation stages must complete in sequence — and once an order is confirmed (finalised), it becomes immutable except for the cancel transition. This forward-only constraint is encoded in the domain entity itself and validated before every state transition.

PaymentIntent (Payment aggregate root). The PaymentIntent aggregate models the lifecycle of a customer's intent to pay. It is created with a specified amount and currency, and transitions through states — Pending, RequiresAction, Processing, Succeeded, Canceled, and Failed — based on gateway interactions. The aggregate tracks payment captures, where each capture debits a portion of the authorised amount. The system enforces the invariant that the sum of all captures must not exceed the original intent amount. A separate payment capture goes through its own state transitions: Succeeded → Captured → Refunded / Voided. The system maintains its own payment state in parallel with the gateway state, enabling consistent behaviour whether using the production Stripe gateway or the development Bogus gateway.

StockItem (Inventory aggregate root). The StockItem aggregate tracks the physical availability of a product variant at a specific warehouse location. It maintains two quantities: on-hand (physical count from warehouse operations) and reserved (units held for active checkouts). The aggregate enforces the invariant that `QuantityOnHand ≥ 0` — stock cannot go negative. Backorder support allows sales beyond on-hand quantity up to a configured backorder limit, but the system tracks the deficit separately. Quantity changes are not performed directly on StockItem; instead, they must be recorded as StockMovement entries in an append-only ledger, preserving a complete and auditable history of every stock change, including the quantity before and after, the reason, and the operating user.

6.2.3 Ubiquitous Language Glossary

A key practice in DDD is establishing a ubiquitous language — a shared vocabulary used by all team members and reflected directly in the codebase. Table Table 14 presents the core terms of the ReSys.Shop domain with their definitions.

Table 14.

Table 14: Ubiquitous Language Glossary: core domain terms, their owning bounded context, and their precise definitions as used throughout the codebase and this thesis.

Term	Context	Definition
Product	Catalog	A product family representing a fashion item concept (e.g., "Cotton T-Shirt"). Holds shared metadata: description, slug, status, fashion-specific attributes, and taxonomy classifications. Does not have a price or SKU directly — those belong to Variants.
Variant	Catalog	A sellable, physical unit of a product (e.g., "Cotton T-Shirt — Red — Large"). Holds SKU, barcode, pricing, inventory tracking flag, and physical dimensions. Each variant belongs to exactly one product.
Master Variant	Catalog	The designated default variant shown on product listing pages. Every product with variants must have exactly one master variant.
Option Type	Catalog	Defines a configurable attribute class such as "Colour", "Size", or "Material". Option types are product-independent and reusable across the catalogue.
Taxonomy / Taxon	Catalog	A hierarchical categorisation tree for organising products. A Taxonomy (e.g., "Department") contains Taxons (e.g., "Clothing" → "Dresses" → "Evening Dresses") forming a nested set structure.
Cart	Ordering	An ephemeral collection of line items that represents a customer's intent to purchase. Carts automatically expire after seven days of inactivity. The cart is the initial state of the Order aggregate.
Line Item	Ordering	A single entry in an order or cart, referencing a specific product variant, quantity, and a price snapshot captured at the time of purchase to insulate historical orders from catalogue price changes.
Checkout State	Ordering	The sequential stage of the purchase process: Address → Delivery → Payment → Confirm → Complete. Progression is forward-only; cancellation is available from any pre-confirmation stage.
Payment Intent	Payment	A data structure representing a customer's intent to pay a specified amount. Follows a defined state machine through the gateway interaction lifecycle.
Payment Capture	Payment	A partial or full debit against a payment intent. Multiple captures can be performed against a single intent (e.g., capturing shipping cost after items ship).
Stock Item	Inventory	The current state of a product variant's availability at a specific warehouse. Maintains on-hand and reserved quantity counters with optimistic concurrency control to prevent overselling.
Stock Movement	Inventory	An immutable ledger entry recording every change to a stock item's quantity — the delta, the balance before and after, the reason, and the operating user. The single source of truth for inventory changes.
Refresh Token	Identity	A long-lived credential used to obtain new short-lived access tokens without re-authentication. Each token is single-use; presenting a previously consumed token triggers revocation of all tokens for that user.

6.2.4 State Machines

Two explicit state machines govern the most critical transactional workflows in the system: the order checkout process and the payment intent lifecycle. Both are encoded in domain entities, validated before every state transition, and drive the sequence of user-facing and system-level actions.

6.2.4.1 Order Checkout State Machine

The order checkout state machine enforces a forward-only progression through five sequential states: Address, Delivery, Payment, Confirm, and Complete. Each state transition is triggered by a specific user action and validated by the domain entity before being committed. Figure Figure 4 depicts this lifecycle.

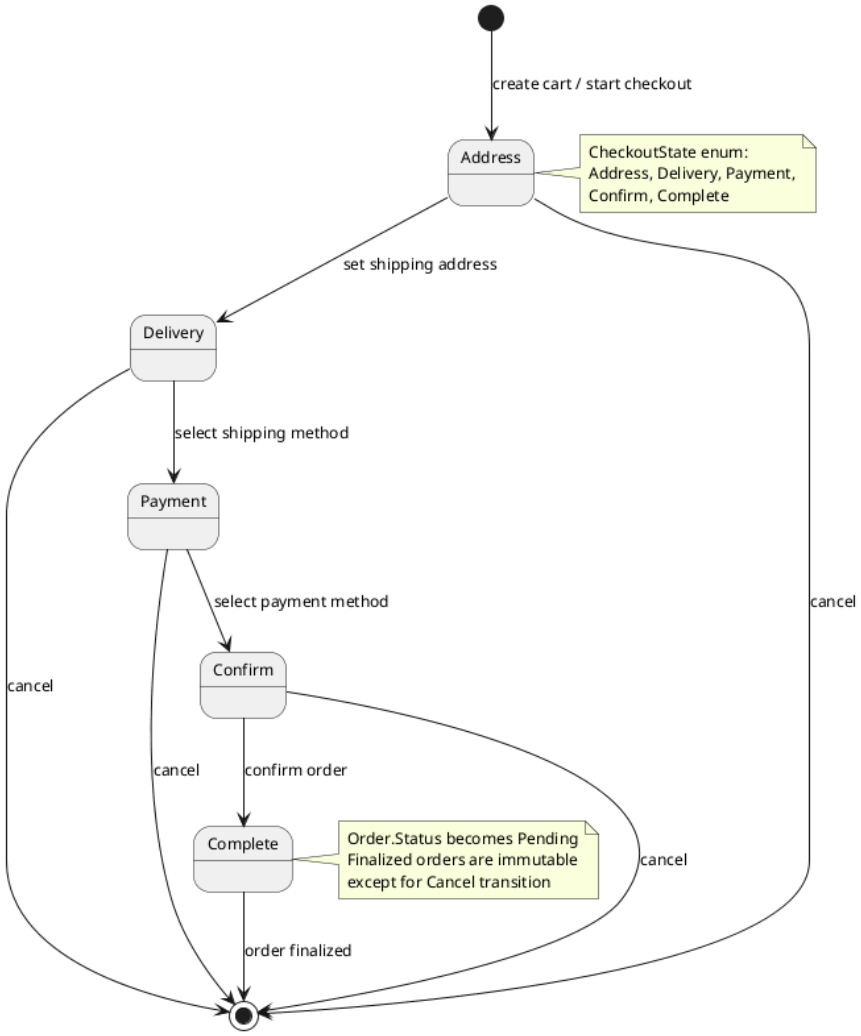

Figure 4.

Figure 4: Order checkout state machine: five sequential states with cancellation available from any pre-confirmation state. The forward-only constraint prevents regressing to earlier checkout stages.

The customer begins by providing a shipping address (Address state), selects a delivery method (Delivery state), chooses a payment method (Payment state), reviews the complete order summary (Confirm state), and finalises the purchase (Complete state). At any point before the Complete state — from Address through Confirm — the customer may cancel the checkout process, which terminates the order without financial consequence.

Once an order reaches the Complete state, it becomes finalised: the order record transitions to Pending status, inventory quantities are reserved for each line item, and the payment intent is processed. From this point forward, the order is immutable except for the cancel transition, which captures the cancellation timestamp and releases reserved inventory. This forward-only design ensures that at every stage of the checkout pipeline, the system can unambiguously determine the customer's position and the next required action.

6.2.4.2 Payment Intent State Machine

The payment intent state machine models the full lifecycle of a payment from creation through to terminal completion, reflecting the state transitions of the Stripe payment gateway while maintaining a parallel system-managed state for offline consistency. Figure Figure 5 shows all states and transitions.

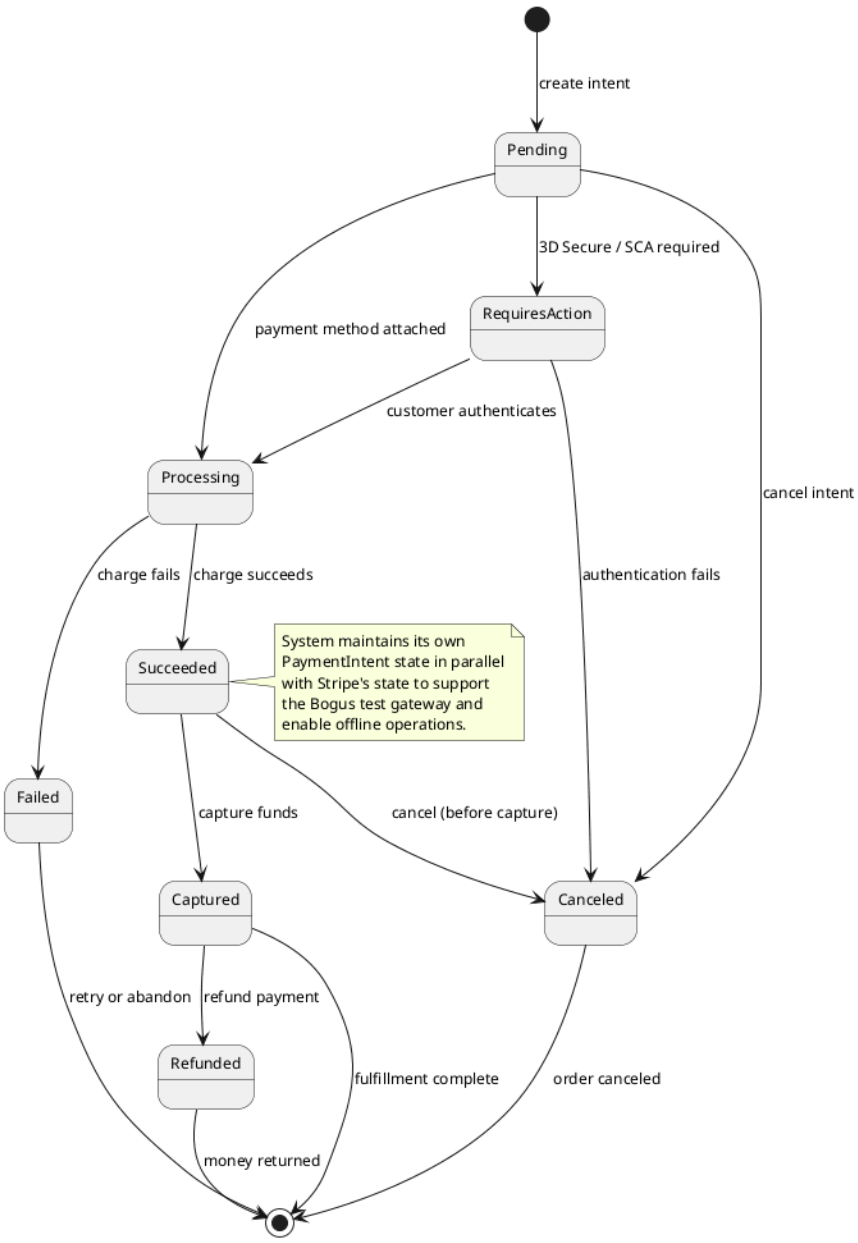

Figure 5.

Figure 5: Payment intent lifecycle: the state machine reflects Stripe gateway states while maintaining a parallel system copy for offline operations and Bogus gateway compatibility. Terminal states are Failed, Canceled, and Refunded.

A payment intent is created in the Pending state. It may transition directly to RequiresAction when the payment requires 3D Secure or Strong Customer Authentication, or to Processing when the payment method has been attached without additional authentication challenges. From RequiresAction, successful customer authentication advances the intent to Processing; failure or timeout moves it to Canceled.

From Processing, a successful charge transitions the intent to Succeeded, while a declined or errored charge moves it to Failed. From Succeeded, the funds may be captured — transferring them from the customer's account — or the intent may be cancelled before capture. A captured intent may later be refunded, returning funds to the customer and reaching the terminal Refunded state.

The system maintains its own copy of the payment state in parallel with the gateway's representation. This design decision serves two purposes. First, it enables the Bogus test gateway — a development-only implementation that simulates payment lifecycles without external calls — to operate against the same domain entities as the production Stripe gateway. Second, it allows the system to reason about payment state offline without querying the gateway API, which improves resilience during network interruptions and reduces external dependency during business operations.

6.3 C4 ARCHITECTURE

The C4 model provides a structured approach to describing software architecture at four levels of abstraction: system context, container, component, and code. This section presents the first three levels for ReSys.Shop, omitting the code-level view as it falls within the scope of the implementation chapter. A deployment diagram complements the C4 views by showing the physical infrastructure.

6.3.1 System Context

The system context diagram positions ReSys.Shop within its environment, showing the human users who interact with the platform and the external systems on which it depends. Figure Figure 6 presents this highest-level view.

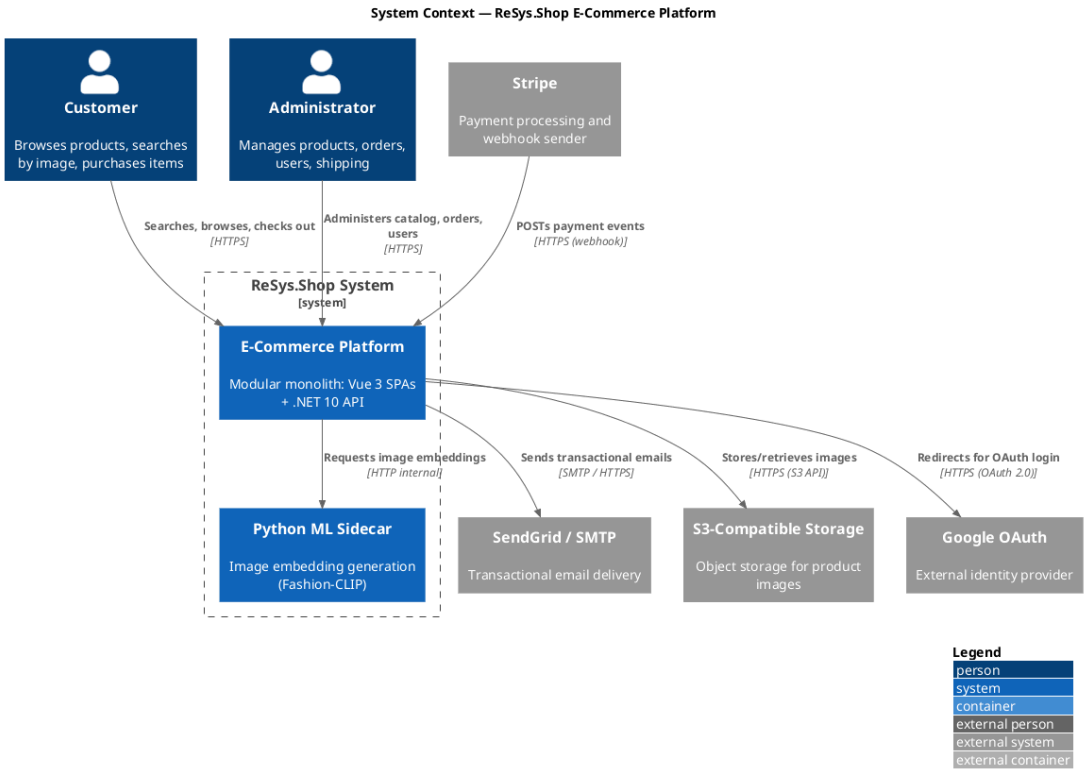

Figure 6.

Figure 6: System Context diagram: ReSys.Shop as a single system boundary with customer and administrator users on one side and external payment, email, storage, identity, and ML services on the other. The modular monolith internally handles all eight business domains within one boundary.

Two categories of human users interact with the platform. Customers browse the product catalogue, perform visual and keyword searches, manage a shopping cart, and complete multi-step checkout — all through the Vue 3 storefront SPA. Administrators manage the full product lifecycle, process orders, monitor inventory levels, and administer user accounts through the Vue 3 admin SPA.

The system depends on five external services. Stripe processes payment intents and sends webhook notifications when payment events occur — the backend validates these webhooks using Stripe's signature verification before acting on them. SendGrid delivers transactional emails such as order confirmations, password reset links, and shipping notifications. An S3-compatible object store persists product images uploaded through the admin interface. Google OAuth provides an alternative authentication path, allowing customers to sign in using their Google credentials. The Python ML Sidecar — deployed as a companion service within the Aspire orchestration boundary — generates image embeddings used by the catalogue's visual search feature.

6.3.2 Container

The container diagram decomposes ReSys.Shop into its deployable units — the processes and data stores that together constitute the running system. Figure Figure 7 presents this view.

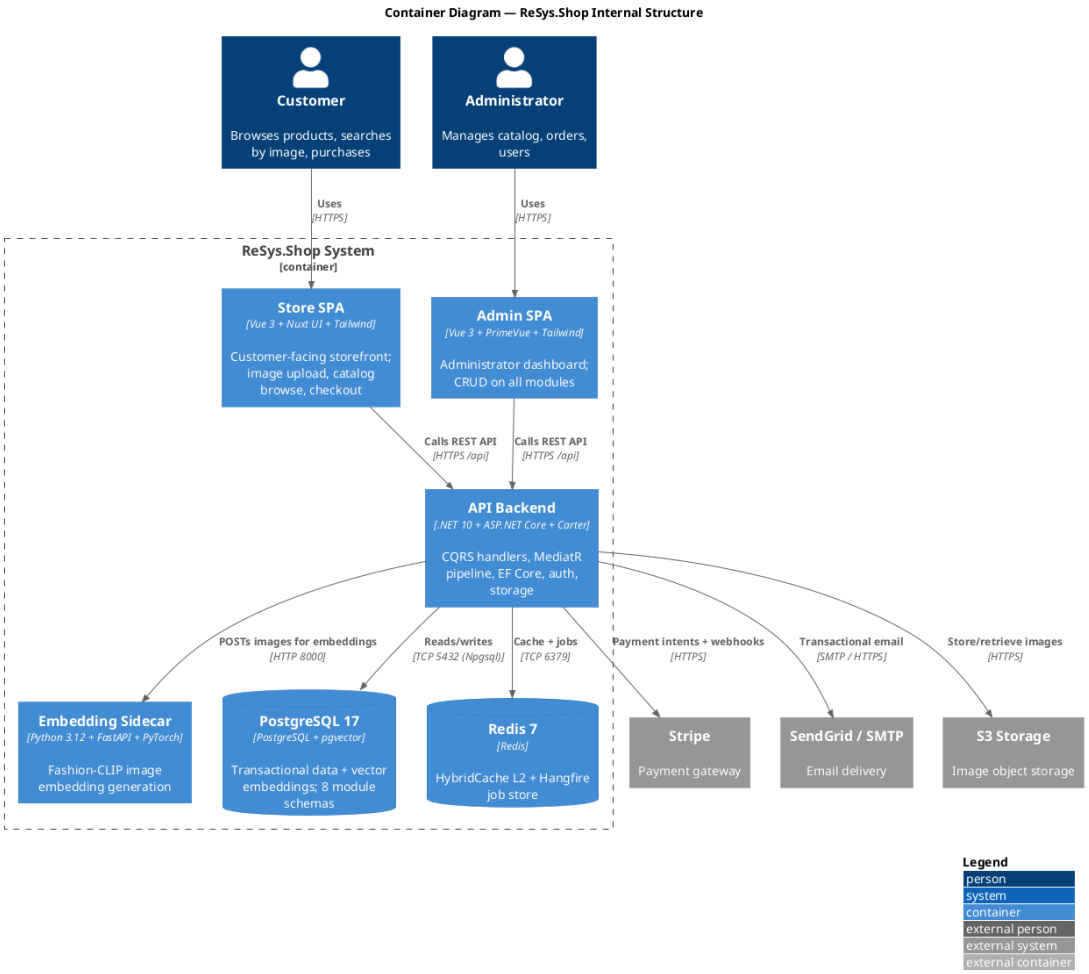

Figure 7.

Figure 7: Container diagram showing the deployable units of ReSys.Shop: two Vue 3 SPAs, the .NET 10 API backend, the Python ML sidecar, PostgreSQL with pgvector, and Redis. Arrows indicate communication protocols between containers.

The system comprises six deployable containers. The Store SPA and Admin SPA — both Vue 3 applications served as static assets — handle all user interface concerns and communicate with the backend exclusively through the REST API over HTTPS. The API Backend — a .NET 10 application running on ASP.NET Core — contains all business logic across eight modules, exposes Carter minimal API endpoints, and orchestrates the MediatR CQRS pipeline for command and query processing. The Embedding Sidecar — a Python 3.12 FastAPI appli-

cation — loads machine learning models into GPU or CPU memory and exposes HTTP endpoints for generating image embeddings.

Two persistent data stores support the platform. PostgreSQL 17 with the pgvector extension serves as the primary database, storing both relational transactional data across eight module-specific schemas and high-dimensional vector embeddings for visual similarity search. Redis 7 fills a dual role: as the second-level distributed cache backing the HybridCache abstraction, and as the persistent job store for Hangfire background job processing — enabling cart expiry, webhook dispatch, and periodic maintenance tasks to survive application restarts.

The communication topology reflects deliberate design constraints. The Vue SPAs call the backend synchronously over HTTPS — never directly accessing the database or external services, which ensures all security policies and data validation are enforced server-side. The backend communicates with PostgreSQL and Redis over internal TCP connections, with the ML sidecar over HTTP on the internal Docker network, and with external services over HTTPS. This design centralises all external integration through the backend container, simplifying security management and operational monitoring.

6.3.3 Component

The component diagram zooms into the API Backend container, revealing its internal structure: the modules, framework services, and cross-cutting concerns that compose the .NET application. Figure Figure 8 presents this view.

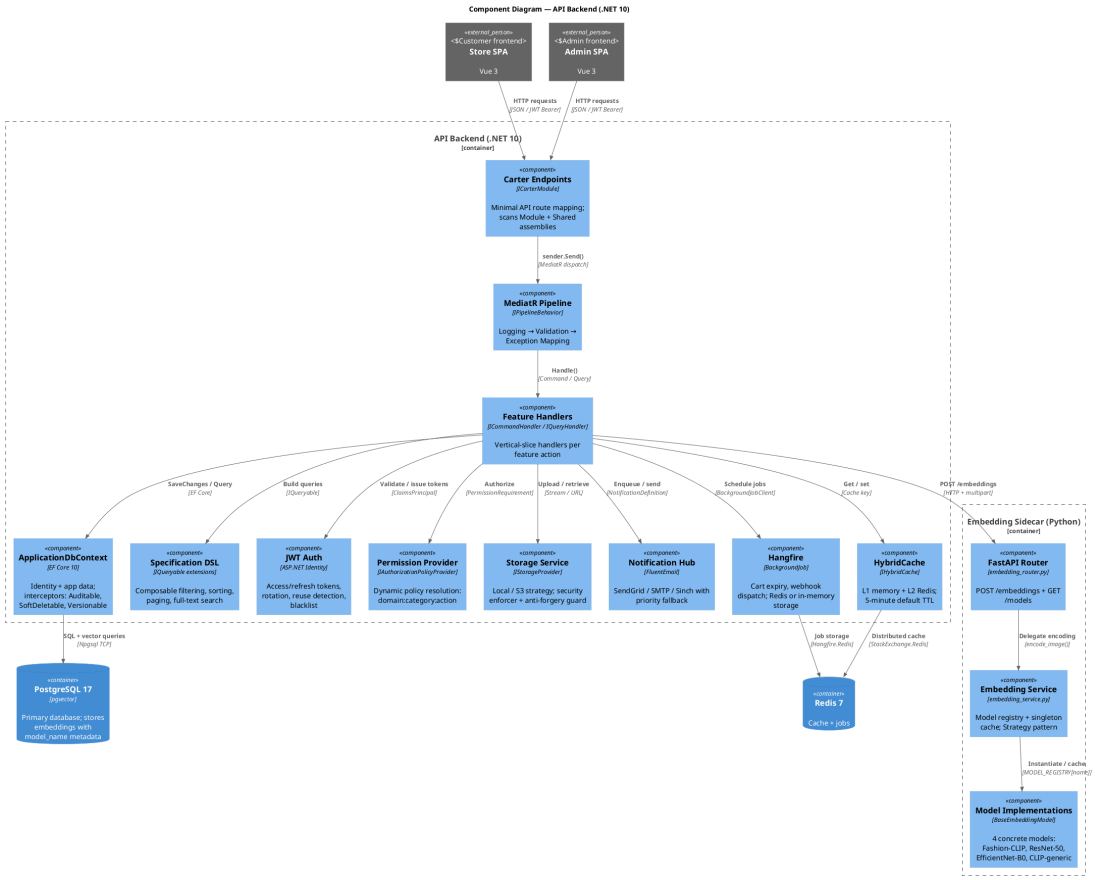

Figure 8.

Figure 8: Component diagram of the API Backend showing the Carter endpoint layer, the MediatR pipeline, the feature handlers, and eight supporting infrastructure components. The Python ML Sidecar is shown with its internal three-layer architecture alongside.

The API Backend is structured as a pipeline. HTTP requests arrive at the Carter endpoints, which are minimal API route groups registered by `ICarterModule` implementations in each module's feature folder. The endpoints are thin — they extract request parameters, dispatch a command or query via `ISender`, and map the `Result<T>` response to an HTTP status code and JSON body. All business logic resides in the feature handlers.

The MediatR pipeline wraps every request with a chain of behaviours: logging captures the request type and timing, validation executes FluentValidation rules before the handler runs, and exception mapping converts unhandled infrastructure failures to standardised problem details. The handlers themselves interact with eight infrastructure components:

- ApplicationDbContext (EF Core 10) with interceptors for auditable timestamps, soft-delete filtering, and row-version concurrency checks.

- A Specification DSL that provides composable `IQueryable` extensions for filtering, sorting, paging, and full-text search — keeping handler code free of query-building boilerplate.
- JWT authentication with ASP.NET Identity, managing access and refresh token issuance, rotation, reuse detection, and token blacklisting.
- A dynamic permission provider that resolves `{domain}:{category}:{action}` permission claims to authorisation policies at runtime without requiring static policy registration.
- A storage service with interchangeable providers (local filesystem or S3-compatible storage) selected via configuration, with built-in file-type validation and anti-forgery guards on uploads.
- A notification hub supporting email (SendGrid/SMTP) and SMS (Sinch) channels with configurable fallback priority.
- Hangfire for background job scheduling and processing — handling cart expiry, webhook dispatch, and periodic health checks.
- HybridCache with two-tier caching: L1 in-memory for sub-millisecond access and L2 Redis for cross-instance consistency.

The Python ML Sidecar follows a three-layer architecture: the FastAPI router handles HTTP request validation and API key authentication, the Embedding Service maintains a singleton model registry with lazy loading and caching, and the model implementations — Fashion-CLIP, ResNet-50, EfficientNet-B0, and generic CLIP — implement a common strategy interface for interchangeable inference backends.

6.3.4 Deployment

The deployment diagram illustrates how the containers map to physical or virtual infrastructure in a production configuration. Figure Figure 9 shows the deployment topology.

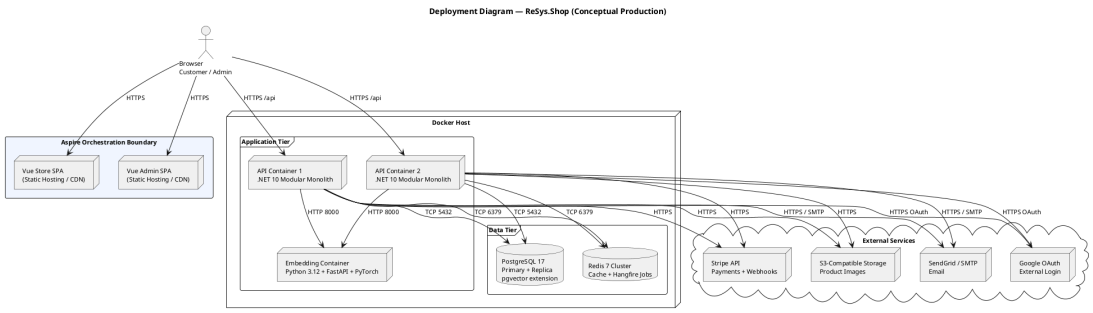

Figure 9.

Figure 9: Deployment diagram showing containerised services within an Aspire orchestration boundary. The API backend is horizontally scalable; the embedding sidecar is stateless; Redis enables distributed state across API replicas.

All services are containerised and orchestrated by .NET Aspire, which manages service discovery, configuration injection, and health monitoring during both development and production deployments. The Vue SPAs are served as static bundles from a CDN or reverse proxy, while the backend services run within Docker containers on a single host or across a cluster.

The API backend is horizontally scalable — multiple container instances share PostgreSQL and Redis, enabling round-robin request distribution. The embedding sidecar is stateless: it loads models into memory on startup, caches them, and serves embedding requests without shared state. Any API instance can call any embedding container. Redis provides the distributed state needed for cache coherence and Hangfire job coordination across API replicas.

PostgreSQL is configured with a primary instance for writes and one or more read replicas for reporting and analytical queries. The pgvector extension is installed on both primary and replicas, enabling vector similarity search from any read path. External services — Stripe, SendGrid, S3 storage, and Google OAuth — are accessed over HTTPS from every API instance, with credentials managed through Aspire's configuration system and never baked into container images.

6.4 DATABASE DESIGN

The ReSys.Shop database is a single PostgreSQL 17 instance organised into per-context schemas, each owned by a bounded context and managed through Entity Framework Core migrations. This section describes the schema organisation, the core entity-relationship model, the pgvector integration for visual search, and the key design decisions that shape the data layer.

6.4.1 Schema Organisation

Each of the eight bounded contexts owns its database schema. The Catalog context manages tables for products, variants, variant images, option types, option values, taxonomies, and taxons. The Ordering context owns orders, line items, adjustments, shipments, and inventory units. The Payment context holds payment intents, payment captures, and payment logs. The Inventory context manages stock locations, stock items, stock movements, and stock reservations. The Identity context — built on ASP.NET Identity Core — manages users, roles, user roles, refresh tokens, and external login providers. The Profile context owns user addresses, wishlists, and notification preferences. The Shipping context manages shipping methods, shipping rates, and shipping zones. The Location context provides country and state reference tables.

Cross-context relationships are implemented through identifier references only — there are no foreign key constraints spanning module boundaries. An

Order references a UserId (Identity context) and a VariantId (Catalog context), but these are stored as UUIDs without database-level referential integrity constraints. This design preserves the logical isolation of each context as a separate schema while keeping all data in a single database, avoiding the complexity of distributed transactions for the most common business operations.

Entity Framework Core manages all migrations from a dedicated Migrations assembly. Each migration is generated by comparing the current database state against the domain entity model, and migrations are applied as part of the application startup or through standalone migration scripts for production deployments.

6.4.2 Core Entity-Relationship Model

Figure Figure 10 presents the entity-relationship diagram for the core business entities across the Catalog and Ordering domains — the two contexts that participate most directly in the visual search and checkout workflows.

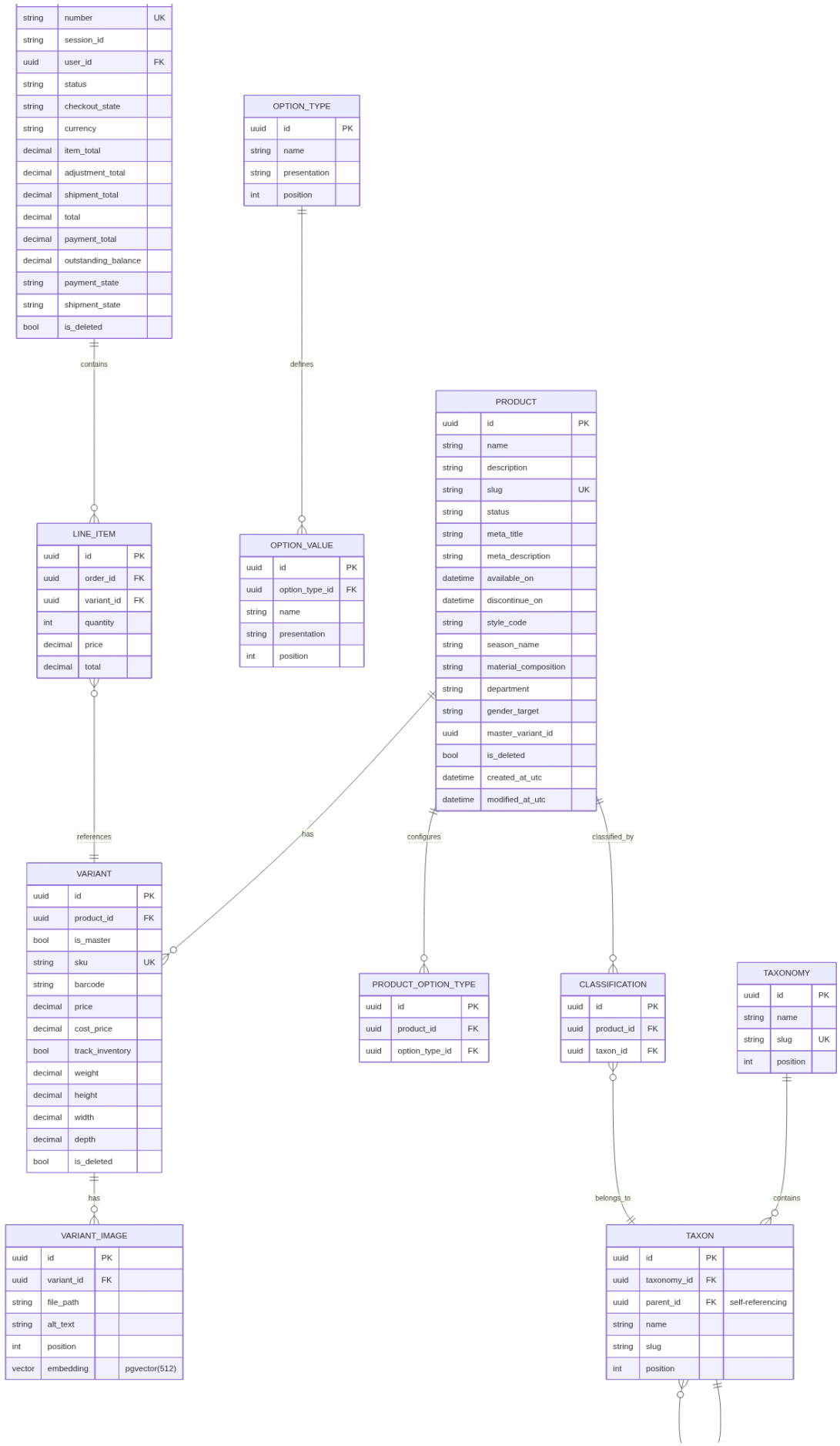
string number UK
string session_id
uuid user_id FK
string status
string checkout_state
string currency
decimal item_total
decimal adjustment_total
decimal shipment_total
decimal total
decimal payment_total
decimal outstanding_balance
string payment_state
string shipment_state
bool is_deleted
contains
OPTION_TYPE
uuid id PK
string name
string presentation
int position
defines
PRODUCT
uuid id PK
string name
string description
string slug UK
string status
string meta_title
string meta_description
datetime available_on
datetime discontinue_on
string style_code
string season_name
string material_composition
string department
string gender_target
uuid master_variant_id
bool is_deleted
datetime created_at_utc
datetime modified_at_utc
LINE_ITEM
uuid id PK
uuid order_id FK
uuid variant_id FK
int quantity
decimal price
decimal total
OPTION_VALUE
uuid id PK
uuid option_type_id FK
string name
string presentation
int position
references
has
configures
classified_by
VARIANT
uuid id PK
uuid product_id FK
bool is_master
string sku UK
string barcode
decimal price
decimal cost_price
bool track_inventory
decimal weight
decimal height
decimal width
decimal depth
bool is_deleted
PRODUCT_OPTION_TYPE
uuid id PK
uuid product_id FK
uuid option_type_id FK
CLASSIFICATION
uuid id PK
uuid product_id FK
uuid taxon_id FK
TAXONOMY
uuid id PK
string name
string slug UK
int position
has
belongs_to
contains
VARIANT_IMAGE
uuid id PK
uuid variant_id FK
string file_path
string alt_text
int position
vector embedding pgvector(512)
TAXON
uuid id PK
uuid taxonomy_id FK
uuid parent_id FK self-referencing
string name
string slug
int position

Figure 10.

Figure 10: Core entity-relationship diagram showing the primary domain entities and their relationships across the Catalog and Ordering bounded contexts. Soft deletion, audit columns, and GUID primary keys are used throughout.

The Catalog domain centres on the Product entity, which has a one-to-many relationship with Variant. Each Variant represents a specific sellable configuration — for example, "Cotton T-Shirt — Red — Large" — with its own SKU, pricing, and inventory tracking flag. Exactly one variant per product is designated as the master variant. Variants relate one-to-many to VariantImages, each of which holds a file path and an optional `embedding` column of type `vector(512)` for pgvector similarity search. Products configure option types — such as Colour or Size — which in turn define option values. Taxonomies contain taxons in a hierarchical structure, with taxons referencing themselves through a parent foreign key to represent tree structures such as "Clothing → Dresses → Evening Dresses". Products are classified by taxons through a many-to-many classification table.

The Ordering domain centres on the Order entity, which has a one-to-many relationship with LineItem. Each line item references a specific variant — not a product directly — because customers purchase specific configurations. The line item captures a price snapshot at the time of purchase, ensuring that historical orders are unaffected by catalogue price changes. Orders are related to users through a UserId reference, optionally nullable to support guest checkout, and track a session identifier for mapping anonymous carts before authentication.

6.4.3 pgvector Integration

PostgreSQL's pgvector extension enables vector similarity search directly within the relational database, eliminating the need for a separate vector database. The `variant_images` table contains an `embedding` column of type `vector(512)` — a fixed-length array of 512 IEEE 754 single-precision floating-point numbers representing the visual features extracted from the image by the ML sidecar.

An HNSW (Hierarchical Navigable Small World) index is created on the embedding column to accelerate approximate nearest-neighbour searches. HNSW provides logarithmic search complexity, enabling sub-10-millisecond queries over tens of thousands of vectors. The index is configured for cosine distance — the recommended distance metric for normalised embeddings from CLIP-family models.

Vector similarity queries use the cosine distance operator (`<=>`), which computes the angular distance between two vectors. A representative conceptual

query pattern is: retrieve all variant images, compute the cosine distance between each stored embedding and the query embedding, order by ascending distance, and return the top results. The system filters results by a configurable minimum similarity threshold — computed as 1 - cosine_distance — defaulting to 0.7, a level at which fashion images typically exhibit perceptible visual similarity.

Each embedding row includes a `model_name` column identifying which ML model generated the vector. This metadata enables per-model filtered queries: when the system switches the active embedding model, only embeddings from that model's columns participate in similarity search, preventing cross-model vector comparisons that would produce meaningless results. The model name also supports the benchmark evaluation in Chapter 6, where multiple models are compared against the same image corpus.

6.4.4 Key Design Decisions

Several design decisions govern the database layer and influence every bounded context:

Universally Unique Identifiers. All primary keys are UUIDs (GUIDs in .NET terminology). This decision eliminates reliance on a central sequence generator, allows safe distributed key generation — useful for client-side offline creation of cart or draft-order records — and prevents the information leakage inherent in auto-incrementing integer primary keys.

Soft Deletion. Most domain entities carry an `IsDeleted` boolean column. Entity Framework Core applies a global query filter that excludes soft-deleted rows from all standard queries. This preserves referential integrity — a deleted product does not orphan its historical orders — while keeping the active working set clean. Deleted entities can be restored by administrators if the deletion was in error.

Audit Columns. Every entity inherits `CreatedAtUtc` and `ModifiedAtUtc` timestamp columns from the `Entity` base class, populated automatically by an EF Core save interceptor. These columns support chronological analysis of catalogue changes, order lifecycle auditing, and debugging data anomalies without external logging correlation.

Composite Indexes. Tables serving high-frequency query patterns carry composite indexes tuned to their access patterns. The Orders table includes `(UserId, Status)` for listing a customer's orders filtered by state, and `(SessionId, Status)` for resolving anonymous carts. These composite indexes avoid the need for separate single-column indexes and reduce disk footprint.

Variable Vector Dimensions. Different embedding models produce vectors of different dimensionalities: 384 for DINOv2-S, 512 for Fashion-CLIP and

CLIP, 768 for DINOv2-B, 1280 for EfficientNet-B0, and 2048 for ResNet-50. PostgreSQL's `vector` type supports this variability — the dimension is part of the column type declaration, and HNSW indexes are built per-dimension. The system stores embeddings from all models in separate rows, each tagged with the model name, and queries against the active model's dimension only.

6.4.5 Per-Context Schema Description

The Identity context stores user accounts with Argon2-hashed passwords, security stamps for session invalidation, and refresh tokens with one-time-use semantics. Role and UserRole tables implement RBAC, while UserLogin records link local accounts to external OAuth providers. UserAddresses store shipping and billing locations with ISO country codes.

The Catalog context's Product table carries fashion-specific metadata columns — style code, season name, material composition, department, and gender target — alongside standard catalogue fields. Variants hold SKU, barcode, cost and selling price in decimal precision, and physical dimensions for shipping calculations. The OptionType and OptionValue tables implement an EAV-lite pattern, allowing administrators to define new product attributes without schema migrations. Taxons use a nested set model with left and right bounds, enabling single-query subtree retrieval for mega-menus and breadcrumb navigation.

The Ordering context stores financial values as decimal with 18-digit precision and 2-decimal scale, avoiding floating-point accumulation errors in tax and discount calculations. Orders maintain separate subtotals for items, adjustments, and shipping, with the total computed as their sum. LineItems capture price snapshots at purchase time, decoupling order history from catalogue changes. OrderHistory provides an append-only audit trail of every state transition.

The Inventory context tracks stock items at a variant-location granularity, with `QuantityOnHand` and `QuantityReserved` maintained as separate counters and protected by optimistic concurrency control using PostgreSQL's `xmin` system column. StockMovements form an immutable ledger — every quantity change, whether from receiving, selling, returning, or stock-taking, is recorded with the delta, balances before and after, unit cost, and an external reference such as an order number or purchase order identifier.

6.5 API DESIGN

The ReSys.Shop API exposes a RESTful interface built on Carter minimal APIs and organised around the MediatR CQRS pattern. This section describes the API architecture, the endpoint organisation scheme, and a summary of the key endpoints that define the platform's external contract.

6.5.1 API Architecture

The API layer acts as a thin orchestration boundary. It contains no business logic; instead, it delegates all processing to the MediatR pipeline. Each request follows a consistent path: the Carter endpoint receives the HTTP request, extracts route and body parameters, constructs a MediatR command or query object, dispatches it through `ISender`, and maps the returned `Result<T>` to an HTTP response. This design keeps endpoints concise — typically six to twelve lines — and concentrates all domain logic in the handler layer, where it is testable without HTTP infrastructure.

Carter modules group related endpoints by module and surface. Each module (Catalog, Ordering, Payment, and so on) registers its own `ICarterModule` implementation, which defines the route groups, HTTP methods, and parameter bindings for that module's endpoints. This modular registration avoids a single monolithic route configuration file and enables each bounded context to own its API surface.

FluentValidation provides input validation through validator classes associated with each command and query. Validators run automatically as part of the MediatR pipeline behaviour, before the handler executes, ensuring that handlers never receive invalid input. Validation failures return standardised `400 Bad Request` responses with field-level error details.

6.5.2 Endpoint Organisation

Endpoints are organised by two dimensions: the business module that owns the operation, and the surface — Admin or Storefront — that serves as the entry point. The URL pattern follows the convention `/api/{module}/{surface}/{action}`, where module identifies the owning bounded context, surface distinguishes administrative from customer-facing operations, and action names the specific operation.

This two-dimensional organisation serves several purposes. It makes the API self-documenting: the URL alone communicates which business area and which user role the endpoint targets. It simplifies authorisation: Admin surface endpoints share a common authorisation policy requiring an administrator role, while Storefront endpoints apply corresponding customer-level policies. And it enables independent versioning: a module can evolve its endpoints without affecting other modules.

Table Table 15 summarises the most architecturally significant endpoints across the platform. These endpoints represent the primary user-facing capabilities — visual search, catalogue browsing, checkout, order history, authentication, and payment — that together define the complete customer and administrator experience.

Table 15.

Table 15: Key API endpoints representing the primary user-facing capabilities of the platform. All endpoints in the Storefront surface serve customer interactions; Admin surface endpoints (not shown) mirror these with full CRUD capabilities on all modules.

Endpoint	Module	Surface	Description
`POST /api/catalog/storefront/search-by-image`	Catalog	Storefront	Accepts an uploaded image file, sends it to the ML sidecar for embedding generation, queries pgvector for the nearest neighbour variant images by cosine similarity, and returns matching products ranked by
`POST /api/catalog/storefront/search-by-intent`	Catalog	Storefront	Re- turns

The admin surface provides full CRUD operations on all module entities — products, variants, orders, inventory, users, shipping methods, and location data — following the same URL pattern with the Admin surface prefix. These endpoints are excluded from the table to maintain focus on the core platform capabilities, but they follow identical architectural patterns: minimal API route groups, MediatR dispatch, FluentValidation, and permission-based authorisation.

6.6 SECURITY DESIGN

The security architecture of ReSys.Shop addresses three layers: authentication — verifying the identity of callers, authorisation — controlling what authenticated callers may do, and hardening — defensive measures against common attack vectors. This section describes each layer in turn.

6.6.1 Authentication

The platform uses JSON Web Tokens (JWT) for bearer token authentication. Upon successful login — via email and password or Google OAuth — the server issues two tokens: an access token with a fifteen-minute lifetime and a refresh token with a longer lifetime. The access token carries the user's identifier, email, and permission claims in a compact signed payload. All authenticated API requests include the access token in the `Authorization` header as a Bearer token.

The refresh token is a long-lived credential stored server-side in the database. When the access token expires, the client presents the refresh token to obtain a new access token and a new refresh token — a pattern known as refresh token rotation. Each refresh token is single-use: upon successful rotation, the consumed token is marked as used and a replacement is issued. If a previously consumed refresh token is presented again — indicating a potential token theft scenario — the system revokes all tokens for that user, forcing re-authentication. This rotation-with-reuse-detection pattern limits the damage window of a compromised refresh token to the interval between rotations.

Guest users — customers who have not yet authenticated — are assigned a session identifier stored in a browser cookie. This session identifier links their anonymous cart to their browsing context and persists across page navigations. Upon registration or login, the anonymous cart is merged with the authenticated user's cart, preserving the shopping intent built during the guest session.

6.6.2 Authorisation

Authorisation is implemented through two complementary mechanisms: role-based access control (RBAC) for broad category restrictions and permission-based claims for fine-grained control.

Roles — such as Customer and Administrator — segregate the Admin and Storefront surfaces. An endpoint in the Admin surface requires the Administrator role; a customer presenting valid credentials without that role receives a `403 Forbidden` response. This coarse check prevents unauthorised access to administrative functions at the infrastructure level, before any business logic executes.

Permissions use a structured claim format: `{domain}:{category}:{action}`. For example, `catalog:products:create` grants permission to create products in the Catalog domain. A dynamic permission provider — `IAuthorizationPolicyProvider` — resolves these claim strings to ASP.NET Core authorisation policies at runtime, eliminating the need for static policy registration for every endpoint. This dynamic resolution enables permission configuration through the database without redeployment: an administrator may create a new role, assign it a set of permission claims, and those permissions take effect across all authorised endpoints immediately.

6.6.3 Security Measures

Several defensive measures harden the platform against common web application attack vectors.

Rate Limiting. Authentication endpoints are rate-limited to five requests per minute per IP address to prevent credential brute-forcing. Registration endpoints are limited to three requests per hour per IP address to deter automated account creation. Payment endpoints are limited to thirty requests per minute to maintain availability during high-traffic checkout events.

Security Headers. All HTTP responses include security headers configured through ASP.NET Core middleware: Content-Security-Policy restricts script and style sources to the application's own domains, HTTP Strict-Transport-Security enforces HTTPS-only connections for a configurable duration, X-Frame-Options prevents the application from being embedded in iframes to block clickjacking, and X-Content-Type-Options prevents MIME-type sniffing by browsers.

File Upload Validation. The visual search and product image upload endpoints enforce strict file validation. Uploaded files undergo magic-byte verification — inspecting the file header bytes rather than trusting the file extension — to confirm they are valid JPEG, PNG, or WebP images. A ten-megabyte size limit prevents resource exhaustion from oversized uploads. Server-side valida-

tion repeats the client-side checks, as client-side validation is a convenience that an attacker can bypass.

Payment Webhook Verification. The Stripe webhook endpoint — which receives payment event notifications — validates each incoming request using Stripe's signature verification algorithm. The webhook payload is hashed with a shared signing secret; if the computed signature does not match the one provided in the Stripe-Signature header, the request is discarded before any business logic processes it. This verification prevents spoofed webhook payloads from injecting fraudulent payment state into the system.

6.6.4 Token Flow

The authentication token lifecycle operates as follows. A client authenticates with email and password, receiving an access token and a refresh token. The access token is short-lived and not stored server-side; it is validated by signature verification and expiration check on each request. When the access token expires, the client sends the refresh token to the refresh endpoint. The server validates the refresh token against the database: if it is valid and has not been used before, the server marks it as consumed, issues a new access token and a new refresh token, and returns both to the client. If the presented refresh token has already been consumed — flagged as used from a previous rotation — the server assumes token theft and revokes all refresh tokens associated with that user, logging the security event. The user must then re-authenticate, which invalidates the compromised token chain and issues fresh credentials. This model provides a self-healing defence against refresh token interception without requiring the user to detect or report the compromise.

6.7 SUMMARY

This chapter has presented the architectural design of ReSys.Shop across six dimensions. The service-oriented system architecture separates presentation (Vue 3), business logic (.NET 10), and machine learning (Python sidecar) into independently deployable services. Domain-Driven Design partitions the business domain into eight bounded contexts communicating through MediatR in-process dispatch, with four architecturally significant aggregate roots enforcing explicit invariants. The C4 model describes the system at context, container, and component levels of abstraction, revealing the communication paths between deployable units and the internal composition of the .NET backend. The PostgreSQL database uses per-context schemas, pgvector for vector similarity search, and a set of consistent design decisions — GUIDs, soft deletion, audit columns — applied across all contexts. The API layer follows the URL convention `/api/{module}/{surface}/{action}` with Carter minimal APIs and MediatR CQRS. The security architecture covers JWT authentication with refresh token rotation,

permission-based authorisation with dynamic policy resolution, and layered defensive measures against common attack vectors. Together, these architectural decisions provide the foundation on which the implementation described in the following chapter is built.

7 IMPLEMENTATION

This chapter describes the concrete realisation of the ReSys.Shop system architecture presented in Chapter 4. The chapter is organised into five sections, progressing from the architectural pattern that structures the codebase, through the machine learning pipeline that constitutes the core research contribution, to the end-to-end visual search flow and its configuration mechanism, and finally a concise survey of the supporting e-commerce modules. The implementation follows the service-oriented design established in the previous chapter: a Vue 3 frontend, a .NET 10 modular monolith backend, and a Python machine learning sidecar, each implemented in the technology most appropriate for its domain.

7.1 VERTICAL SLICE ARCHITECTURE

The .NET backend is organised according to **Vertical Slice Architecture** (VSA) — a pattern in which code is grouped by business capability rather than by technical layer. In a traditional layered architecture, all data access classes reside in one project, all business logic in another, and all web controllers in a third. VSA inverts this convention: all code required to fulfil a single feature — the request, the handler, the response, the validator — is co-located within one directory. This organisation makes the system easier to navigate because a developer tracing a feature through the codebase follows a vertical path rather than jumping across four or five horizontal layers.

The implementation uses Carter minimal APIs for route registration and MediatR for command and query dispatch. Each bounded context — Catalog, Ordering, Payment, and the remaining five modules — contributes its own `ICarterModule` implementation, which registers all endpoints belonging to that context. Endpoints are thin: they extract parameters from the HTTP request, construct a MediatR command or query object, dispatch it through `ISender`, and map the resulting `Result<T>` to an HTTP status code and JSON body. All business logic, database access, and domain invariant enforcement reside in the handler layer, where they are testable without HTTP infrastructure.

FluentValidation ensures that every command and query object is validated before its handler executes. Validation classes live alongside the handlers they protect, keeping the validation rules visible and colocated with the logic they guard. The MediatR pipeline wraps each request with three behaviours — validation, logging, and exception mapping — applied automatically to every handler in the system. A fourth pipeline behaviour enforces feature-level trans-

action boundaries, ensuring that handlers operating on multiple database rows do so within a single unit of work.

The vertical slice organisation enforces a strict module boundary rule: no bounded context may import another context's namespace. All inter-module communication flows through MediatR's `ISender` interface. A context may dispatch a query to another context or publish a notification that other contexts react to, but it may never reference a class, type, or namespace belonging to another module. This constraint preserves the logical isolation of the bounded contexts — the same isolation that would be enforced by network boundaries in a microservices architecture — while retaining the deployment simplicity of a single process.

7.2 ML EMBEDDING PIPELINE

The Python machine learning sidecar is the core research contribution of this thesis. It is a specialised AI service responsible for generating high-dimensional vector embeddings from product images, exposing endpoints consumed by the .NET backend through HTTP. This section describes the internal architecture, the model loading strategy, the embedding generation flow, and the health monitoring mechanism.

7.2.1 Service Architecture

The ML sidecar is built on Python 3.12 with the FastAPI framework and runs under the Uvicorn ASGI server. Its internal architecture follows a three-layer design:

- The **FastAPI interface layer** handles HTTP concerns: routing requests to the correct endpoint, validating the API key presented in the `X-API-Key` header, parsing multipart form data containing image bytes, and serialising embedding results to JSON. This layer contains no machine learning logic.
- The **Model Manager layer** is a singleton service responsible for the lifecycle of deep learning models. It maintains a registry of loaded models, loads new models on demand, caches them in GPU or CPU memory, and dispatches inference requests to the correct model instance. The singleton pattern ensures that a single copy of each model — typically consuming several gigabytes of VRAM — serves all inbound requests.
- The **PyTorch Runtime layer** is the hardware-facing component that executes forward passes through the neural network. It handles device placement — selecting CUDA for NVIDIA GPUs, MPS for Apple Silicon, or CPU as a fallback — and manages the tensor operations that convert raw image data into embedding vectors.

The service exposes two endpoints: `POST /embeddings`, which accepts image bytes and returns a 512-dimensional float vector, and `GET /health`, which reports the operational status of the underlying hardware and loaded models. The service is containerised as a Docker image and orchestrated by .NET Aspire, which manages service discovery — the backend addresses the ML sidecar as `http://ml-service` on the internal Docker network — and health-check restarts.

7.2.2 Model Loading Strategy

The ML sidecar employs a **lazy loading** strategy to optimise resource utilisation. Deep learning models are not loaded at service startup, when they would consume gigabytes of GPU memory before the first request arrives. Instead, models are loaded upon their first invocation — when the .NET backend sends an image to be embedded for the first time, the Model Manager instantiates the configured model, loads its weights into memory, and caches the instance for all subsequent requests.

The Model Manager implements the **Strategy Pattern**: each model conforms to a common interface with a standard `generate_embedding(image_bytes)` method, and the manager selects the active model based on the `EMBEDDING_MODEL` environment variable. Changing models requires only a configuration change and a service restart — no code changes. The Model Manager maintains an internal dictionary-style cache keyed by model name, so a request for a previously loaded model returns the cached instance immediately.

The supported models span four architectural families. **Convolutional neural networks** are represented by ResNet-50 (2048-dimensional output) and EfficientNet-B0 (1280-dimensional), both pre-trained on ImageNet and serving as lightweight feature extractors. **Vision transformers** include DINOv2 ViT-S/14 (384-dimensional) and DINOv2 ViT-B/14 (768-dimensional), leveraging self-supervised pre-training that requires no labelled data. **CLIP-based models** — CLIP ViT-B/32, CLIP ViT-B/16, and CLIP ViT-L/14 — produce 512-dimensional embeddings in a shared text-image latent space, enabling multimodal search where a text query can find visually similar products. **Fashion-specific models** are represented by Fashion-CLIP, a CLIP variant fine-tuned on over 700 000 fashion images with domain-specific vocabulary, also producing 512-dimensional vectors.

7.2.3 Embedding Generation Flow

The embedding generation process follows a precise pipeline from image reception to vector output. Figure Figure 11 illustrates this flow.

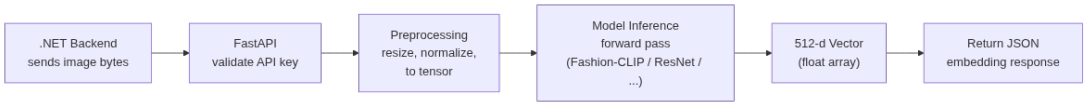

Figure 11.

Figure 11: ML embedding pipeline: the step-by-step flow from image bytes received by the FastAPI interface to a 512-dimensional float vector returned as JSON to the .NET backend.

The pipeline comprises seven sequential steps. The .NET backend initiates the process by sending raw image bytes to the `POST /embeddings` endpoint. The FastAPI interface validates the `X-API-Key` header against the expected key and rejects unauthorised requests with a 401 response. The Model Manager retrieves the currently configured model from its cache — loading it from disk on the first request — and dispatches the image payload to the model's preprocessing pipeline.

The preprocessing stage normalises the input image to the format expected by the selected model. The image is resized to the model's expected input dimensions — 224 by 224 pixels for most architectures — and converted from interleaved colour channels to separated channel tensors. ImageNet-normalisation statistics are applied: each colour channel is centred to the ImageNet mean and scaled by the standard deviation, matching the distribution on which the model was originally trained.

The preprocessed tensor is passed through the model's forward pass. For convolutional models, this involves a cascade of convolution, activation, and pooling operations that extract hierarchical features. For transformer-based models, the image is divided into patches, each patch is projected into a token embedding, and self-attention layers compute contextual relationships between patches across the entire image. The output of the forward pass is pooled — typically via global average pooling — to produce a single fixed-length vector regardless of the original image size.

The resulting vector is a 512-dimensional array of single-precision floating-point numbers (for CLIP-family models; other dimensionalities for other architectures). This vector encodes the visual essence of the image — colour palette, texture patterns, silhouette shape, and semantic category — in a compressed numerical form suitable for similarity computation. The vector is serialised to a JSON array and returned to the .NET backend as the response body, together with metadata fields identifying the model name and generation time in milliseconds.

7.2.4 Health Monitoring

The `/health` endpoint provides a comprehensive operational status for the Aspire orchestration layer. It verifies three conditions: that the GPU environment is active and accessible, confirming CUDA or MPS availability; that the configured model is successfully loaded into memory and ready for inference; and that a baseline inference pass completes without errors, validating the entire pipeline from preprocessing to output generation. The health response includes diagnostic data — model load status, last inference time, GPU memory utilisation — enabling the orchestrator to detect degraded states and trigger Docker restart policies automatically without human intervention.

7.3 CBIR SEARCH FLOW

The content-based image retrieval search flow is the primary user-facing capability enabled by the ML embedding pipeline. It spans four system components — the Vue 3 storefront, the .NET API backend, the Python ML sidecar, and PostgreSQL with pgvector — and must complete in under one second to remain viable as a real-time feature. Figure Figure 12 depicts the full cross-service interaction.

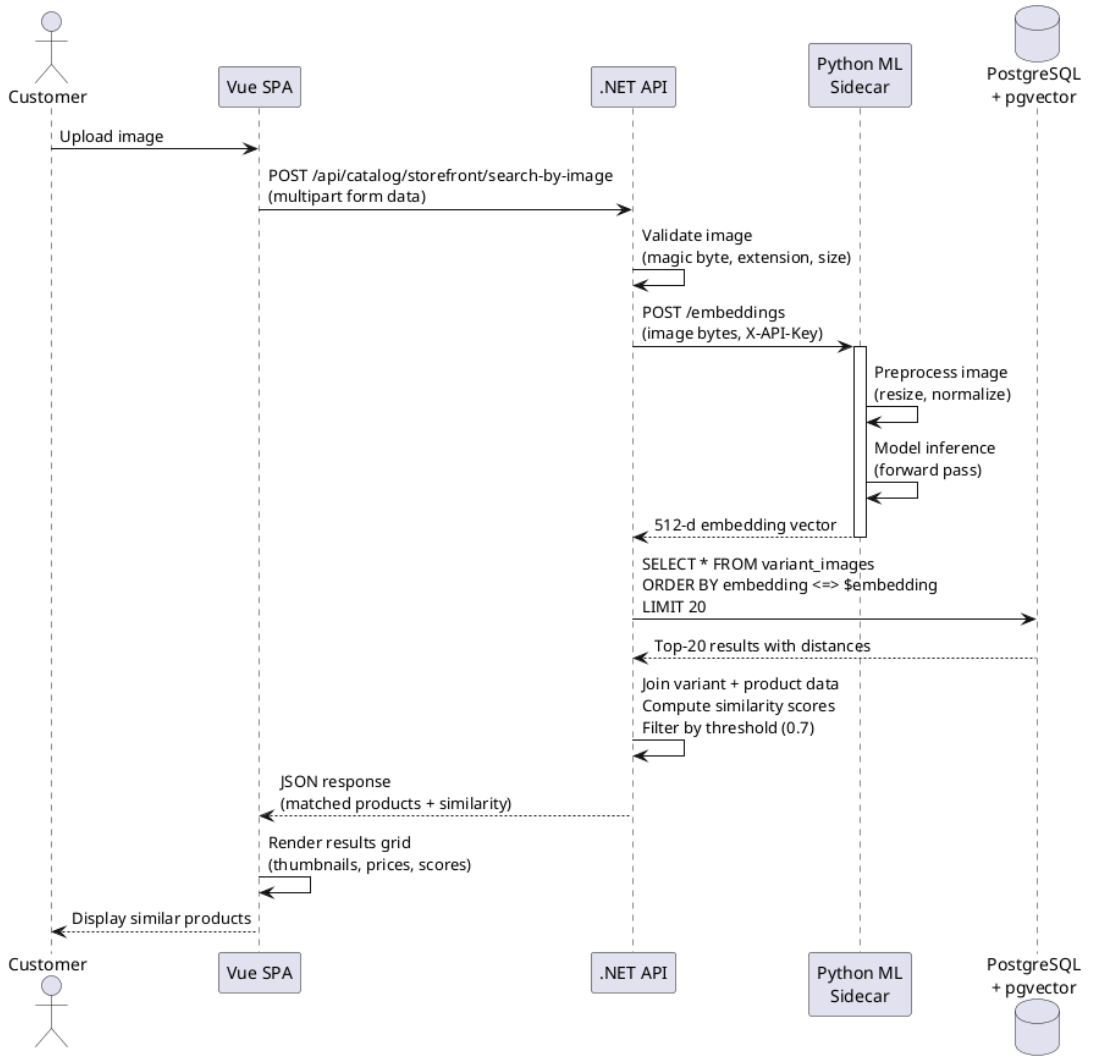

Figure 12.

Figure 12: CBIR search sequence: the complete end-to-end flow from customer image upload through embedding generation and vector search to ranked results display, spanning the Vue frontend, .NET API, Python ML sidecar, and PostgreSQL pgvector.

The flow begins when a customer uploads an image through the Vue storefront interface. The client performs an initial validation pass — checking that the file type is JPEG, PNG, or WebP and that the file size does not exceed ten megabytes — and presents immediate feedback if the upload fails these checks. Upon passing client-side validation, the image is sent as a multipart form data request to the `POST /api/catalog/storefront/search-by-image` endpoint on the .NET backend.

The backend performs a second, stricter validation pass. It verifies the file's magic bytes — the binary header signature of a valid image file — rather than trusting the file extension, which a malicious client could forge. It confirms the MIME type and enforces the same ten-megabyte size limit server-side. If

validation passes, the backend sends the raw image bytes to the Python ML sidecar's `/embeddings` endpoint with the `X-API-Key` header.

The ML sidecar executes the embedding pipeline described in Section 5.2.3 — preprocessing, model inference, vector output — and returns a 512-dimensional float vector together with the model name metadata. The backend now holds a numerical representation of the customer's image that can be compared against every product image in the catalogue.

The vector search query executes against the `variant_images` table in PostgreSQL. The query computes the cosine distance between the query embedding and every stored embedding using the `<=>` operator provided by the pgvector extension, orders results by ascending distance, and returns the closest matches. An HNSW index on the embedding column reduces the search complexity to logarithmic time — sub-ten milliseconds on a corpus of tens of thousands of images. The query includes a `model_name` filter, ensuring that only embeddings generated by the currently active model participate in the comparison; cross-model vector comparisons would produce meaningless results because different models embed images into different latent spaces.

The backend assembles the results into a search response. It joins the top-matching variant image rows with their parent variant and product records to retrieve titles, pricing, and thumbnail URLs. It converts cosine distances into similarity scores — computed as one minus the distance — and filters out results below a configurable minimum similarity threshold, defaulting to 0.7. In fashion image retrieval, a cosine similarity of 0.7 typically corresponds to perceptible visual similarity, while values above 0.9 indicate near-identical items. A product may appear multiple times in the results if multiple of its variant images matched the query, but the response deduplicates by product and returns the highest-scoring match per product.

The backend returns a JSON array of matching products to the Vue storefront. Each entry includes the product title, the matched variant's thumbnail URL, the selling price, the similarity score expressed as a percentage, and a product URL for navigation. The Vue frontend renders these results as a grid of product cards with thumbnail images, price labels, and similarity score badges, completing the search experience. The total end-to-end latency — from upload to rendered results — is designed to remain under one second, with the model inference typically accounting for fifty to one hundred milliseconds of that budget depending on the active model.

7.4 MODEL CONFIGURATION

The ability to switch between embedding models without code changes is a key implementation decision that supports the benchmark evaluation presented in Chapter 6. The mechanism centres on a single environment variable —

`EMBEDDING_MODEL` — that controls which model the ML sidecar loads at first request.

The environment variable accepts a short model identifier: `fashion_clip` for the fine-tuned fashion model, `resnet50` for the classic CNN baseline, `efficientnet_b0` for the lightweight scaled CNN, `clip_vit_b32` or `clip_vit_b16` or `clip_vit_l14` for general-purpose CLIP variants, and `dinov2_vits14` or `dinov2_vitb14` for self-supervised vision transformers. When the Model Manager receives its first embedding request, it reads this variable, instantiates the corresponding model class, loads weights from disk, and caches the instance. Subsequent requests are served from the cache. Changing the active model requires updating the environment variable and restarting the container — a configuration-only change with no code modification.

Each stored embedding carries two metadata columns: `model_name`, identifying which model generated the vector, and `model_version`, capturing the weight snapshot used. When the backend queries pgvector for similar images, it filters by `model_name` — only embeddings from the currently active model participate in the search. This filter ensures that the system can store embeddings from multiple models simultaneously, with each model's embeddings occupying a separate conceptual index within the same physical column. After switching the active model, newly uploaded images begin generating embeddings under the new model name, while existing embeddings under the old model name remain available but are excluded from search queries.

This configuration mechanism enables the benchmark workflow that Chapter 6 depends on. Eleven models are evaluated against the same image corpus by loading each model in sequence, generating embeddings for the full query and catalogue image sets, running the retrieval evaluation protocol, and recording the results — all without modifying application code. In production, the same mechanism enables A/B testing: two instances of the ML sidecar, each configured with a different model, serve embedding requests to different user cohorts. An administrator can compare the business metrics of the two cohorts — click-through rates, conversion rates, time-to-purchase — to determine which model produces the best real-world search experience.

7.5 E-COMMERCE CORE

The preceding sections focused on the visual search pipeline, which constitutes the research contribution of this thesis. This section provides a concise survey of the e-commerce platform modules that surround and support that pipeline. Each module is given a single paragraph below; none is the primary subject of this thesis, but together they form the functional platform within which the visual search feature operates.

Catalog Management. Products are created with fashion-specific metadata fields — style code, season name, material composition, department, and gender target — alongside standard catalogue attributes such as title, description, and publication status. Each product may define variants — sellable configurations combining a size and colour, each with its own SKU, barcode, independent pricing, and physical dimensions — with exactly one variant designated as the master variant displayed on listing pages. Product images support multiple variants with automatic thumbnail generation, and each image may store an embedding vector generated by the ML sidecar. Hierarchical taxonomies organise products into a nested category tree — for example, Clothing, Dresses, Evening Dresses — enabling faceted navigation through the catalogue browser.

Order Processing. The ordering module orchestrates the complete customer purchase lifecycle. Carts — the initial state of the Order aggregate — auto-expire after seven days of inactivity through a Hangfire background job that periodically cleans up abandoned carts. Checkout progresses through the five-state forward-only state machine described in Chapter 4: Address, Delivery, Payment, Confirm, and Complete. Order numbers are generated inside database transactions with RepeatableRead isolation to prevent duplicate numbering under concurrent checkout loads, a subtle but critical correctness measure in high-traffic flash-sale scenarios. Each order records the checkout state alongside payment and shipment sub-states tracked independently, enabling partial fulfilment — a customer's order may ship in two batches as inventory becomes available.

Payment Handling. The payment module manages the lifecycle of payment intents — the customer's commitment to pay a specified amount — through the state machine presented in Chapter 4. Two gateway providers exist: the Stripe gateway handles real payment processing with webhook signature validation using Stripe's signing secret and built-in idempotency keys that prevent double-charging during network retry scenarios; the Bogus gateway simulates the complete payment lifecycle for development environments, automatically transitioning through states without external API calls. The system maintains its own copy of the payment state in parallel with the gateway's representation, enabling offline access to payment status and consistent behaviour across both gateway implementations.

Inventory Tracking. Stock quantities are tracked per variant per warehouse location with two separate counters: on-hand quantity, representing the physical stock count, and reserved quantity, representing units held for active checkouts. The invariant `QuantityOnHand >= 0` is enforced at the domain entity level, preventing negative stock states. Stock reservations are acquired through atomic transactions using row-level pessimistic locking — a `SELECT FOR UPDATE` that serialises concurrent access to the same inventory row. When two customers attempt to purchase the last unit of a product, the database lock ensures sequen-

tial execution, and the second request receives a graceful out-of-stock rejection. Every stock change — whether from receiving, selling, returning, or stocktaking — is recorded as a StockMovement entry in an append-only ledger with the quantity before and after, the reason, and the operating user.

Background Automation. Hangfire serves as the background job processor, executing scheduled and on-demand tasks that are not part of the synchronous request-response cycle. Cart expiry jobs run on a twenty-minute interval, removing carts with no activity for seven days. Embedding generation retries handle transient failures when the ML sidecar is briefly unavailable — failed embedding requests are queued for retrial with exponential back-off. Periodic index maintenance rebuilds or optimises HNSW indices on the vector column to keep nearest-neighbour search performance stable as the catalogue grows. All jobs are persisted in Redis, ensuring that scheduled work survives application restarts and container reorganisations.

8 TESTING & EVALUATION

This chapter presents the evaluation of the ReSys.Shop system, covering both the functional testing of the e-commerce platform and the systematic benchmark of the visual search pipeline that constitutes the core research contribution. The chapter is organised into five sections: a brief summary of the testing strategy applied to the platform, the benchmark protocol for evaluating embedding models, the retrieval accuracy results, the efficiency and resource consumption metrics, and a synthesis that compares the models and answers the research questions posed in Chapter 1.

8.1 TESTING STRATEGY

The ReSys.Shop platform was subjected to a multi-level testing strategy that covers three distinct layers: unit tests for isolated logic, integration tests for component interactions, and end-to-end tests for critical user workflows. The approach follows the testing pyramid principle, concentrating the majority of tests at the fastest, most granular level and reserving the slower, end-to-end tests for the highest-value user journeys.

8.1.1 Unit Testing

Unit tests form the foundation of the verification strategy. The .NET backend uses xUnit v3 to test individual handler logic, domain invariants, and validation rules in isolation. Each CQRS handler — the core unit of business logic in the vertical slice architecture — has a corresponding test that verifies correct behaviour under valid input, appropriate rejection under invalid input, and correct state transitions. Domain invariants such as the order state machine transitions and the inventory non-negative stock constraint are enforced through unit tests

that execute without any external dependencies. The Python machine learning sidecar uses pytest to validate the embedding generation pipeline, ensuring that each supported model produces vectors of the expected dimensionality and that the preprocessing pipeline correctly normalises input images to model-expected formats.

8.1.2 Integration Testing

Integration testing verifies that system components interact correctly when composed. Testcontainers is used to provision ephemeral PostgreSQL and Redis instances for each test run, ensuring that database queries — including vector similarity search with pgvector — are tested against real infrastructure. Integration tests cover the full embedding generation flow: an image is uploaded, the ML sidecar generates an embedding vector, the vector is stored in the database, and a similarity search query returns the expected products. The cross-service communication between the .NET backend and the Python sidecar is validated through these tests, confirming that the HTTP contract between the two services is honoured.

8.1.3 End-to-End Testing

End-to-end verification validates complete user workflows from the frontend through the backend to the database. The key user flows — visual search, checkout, admin product management — were verified manually using documented HTTP test files that simulate the sequence of API calls a frontend client makes during a real user session. Automated end-to-end testing via Playwright covers the most critical paths: the visual search flow from image upload to results display, and the checkout flow from cart addition to order confirmation. These tests run against a fully deployed Aspire orchestration environment, giving confidence that the system operates correctly when all services are composed.

8.2 BENCHMARK PROTOCOL

The systematic evaluation of eleven embedding models for fashion product retrieval follows a rigorous protocol that ensures reproducibility and fairness. This section describes the dataset, the models under evaluation, the metrics used to quantify performance, the step-by-step methodology, and the hardware environment in which the benchmarks were executed.

8.2.1 Dataset

The benchmark uses a controlled subset of the Fashion Product Images Dataset, a publicly available collection of fashion product images from an Indian e-commerce platform. The subset consists of 5,000 catalogue images spanning

five product categories: tops (1,500 items), bottoms (1,200 items), footwear (1,000 items), accessories (800 items), and jewellery (500 items). This balanced distribution prevents class imbalance from skewing retrieval metrics — a model that consistently retrieves items from the largest category would appear deceptively accurate under an unbalanced dataset.

Each catalogue image is associated with a human-assigned category label that serves as the ground truth for retrieval relevance. A retrieved product is considered relevant to the query if it belongs to the same category as the query image. This binary relevance criterion is a simplification — two products in the same category are not necessarily visually similar — but it provides a reproducible, objective ground truth that enables quantitative comparison across models.

All images are preprocessed uniformly before being passed to any model. Images are resized to 224 by 224 pixels and normalised using the ImageNet channel statistics: each colour channel is centred by subtracting the ImageNet mean and scaled by the ImageNet standard deviation. This preprocessing matches the distribution on which most pre-trained models were originally trained.

8.2.2 Models Evaluated

The benchmark evaluates eleven pre-trained embedding models spanning four architectural families. Table Table 16 summarises the models grouped by their architecture type.

Table 16.

Table 16: Eleven embedding models evaluated in the benchmark, grouped by architecture family. All models use pre-trained weights from public model repositories.

Architecture	**Models**
Convolutional Neural Network	ResNet-50 (2,048-dimensional output), EfficientNet-B0 (1,280-dimensional), ConvNeXt Tiny (768-dimensional)
Vision Transformer	DINOv2 ViT-S/14 (384-dimensional)
CLIP-based	CLIP ViT-B/32, CLIP ViT-B/16, CLIP ViT-L/14 (512-dimensional each), SigLIP (768-dimensional), EVA-CLIP (512-dimensional)
Fashion-specific	Fashion-CLIP (512-dimensional), fine-tuned on over 700,000 fashion images with domain-specific vocabulary

The eleven models span a wide range of architectural approaches. Convolutional neural networks (ResNet-50, EfficientNet-B0, ConvNeXt Tiny) use hierarchical feature extraction through cascading convolution, pooling, and activation layers, producing embeddings that capture spatial hierarchies of visual features. Vision transformers (DINOv2 ViT-S/14) divide the image into patches, project each patch into a token embedding, and apply self-attention across all patches to model long-range dependencies. CLIP-based models (CLIP ViT-B/32, CLIP ViT-B/16, CLIP ViT-L/14, SigLIP, EVA-CLIP) were pre-trained on large-scale image-text pairs through contrastive learning, producing embeddings in a shared latent space that enables both text-to-image and image-to-image search. Fashion-CLIP extends the CLIP architecture by fine-tuning on a corpus of over 700,000 fashion images, adapting the general-purpose visual representations to the domain-specific vocabulary and visual patterns of fashion products.

Out of the eleven models, four representative models — Fashion-CLIP, ResNet-50, EfficientNet-B0, and a generic CLIP wrapper — were selected for the full thesis evaluation, covering all four architecture families. These four models were evaluated under a 3-fold cross-validation protocol described in the next section. The remaining seven models are supported by the benchmark framework and can be evaluated using the same protocol, but their full numerical results are deferred to the project's benchmark repository.

8.2.3 Metrics

Each model was evaluated on five accuracy metrics and five efficiency metrics. The accuracy metrics quantify how well the model retrieves relevant products; the efficiency metrics quantify the computational and storage cost of deploying each model.

Mean Average Precision (mAP) is the primary accuracy metric. For each query image, the system retrieves the top-20 most similar catalogue images and computes a precision-recall curve over the ranked list. The area under this curve is the average precision (AP) for that query. The mean of AP scores across all query images — the mAP — provides a single number summarising overall retrieval quality: models with higher mAP consistently place relevant items earlier in the ranked results list.

Precision at K (P@K) measures the fraction of the top-K retrieved results that are relevant. For example, P@10 = 0.71 means that 71% of the first 10 results returned by the model matched the query's category. Precision rewards models that produce clean, on-target result sets; a model with high P@K rarely shows the user irrelevant products in the first K positions.

Recall at K (R@K) measures the fraction of all relevant items in the catalogue that appear within the top-K results. R@10 = 0.40 means that 40% of

all items in the query's category were found among the first 10 retrieved results. Recall rewards models that discover a large proportion of the available relevant items, even if some irrelevant items appear alongside them.

Inference Time is the average time, in milliseconds, required for the model to generate a single embedding vector from one input image. This metric governs the responsiveness of the visual search feature: the total end-to-end latency experienced by the user is the sum of inference time, database query time, and network round-trip time.

Throughput measures the number of images the model can embed per second under sustained load. Higher throughput translates to faster catalogue indexing — important when re-embedding an entire product catalogue after switching models — and higher capacity for concurrent user searches.

Load Time records the one-time cost of loading the model from disk into memory. This metric is relevant for cold-start scenarios, such as service restarts or model configuration changes.

Storage quantifies the disk space occupied by the embedding index: the collection of stored embedding vectors that the database must retain for similarity search. Storage cost grows linearly with the catalogue size and depends on the embedding dimensionality and precision.

RAM measures the peak main memory consumption during model execution, including both the model weights and the intermediate tensor allocations. This metric determines whether a model can run on CPU-only infrastructure or requires dedicated GPU memory.

8.2.4 Methodology

Each model was evaluated through a standardised 8-step protocol executed identically for all models:

1. Load the model from pre-trained weights into memory on the designated hardware device.
2. Generate an embedding vector for every query image in the dataset.
3. Generate an embedding vector for every catalogue image in the dataset.
4. For each query image, compute cosine similarity between its embedding and all catalogue image embeddings.
5. Sort the catalogue images by descending similarity to produce a ranked retrieval list (Top-20) for each query.
6. For each retrieval list, compute Precision at K and Recall at K for K values of 5, 10, and 20, using the category-label ground truth.
7. Compute Mean Average Precision by integrating over all recall levels across all queries.

8. Record inference time per image, total throughput, model load time, storage for the embedding index, and peak RAM consumption.

Steps 1-8 were repeated for each of the four evaluated models. The evaluation used 3-fold cross-validation: the dataset was partitioned into three stratified folds, each preserving the original category distribution. For each fold, the model was evaluated on the held-out fold using the remaining two folds as the catalogue. The reported metrics are the mean and standard deviation across the three folds, providing both point estimates and measures of variability.

8.2.5 Hardware Environment

All benchmarks were conducted on a standard development workstation to represent a realistic deployment scenario typical of small-to-medium e-commerce operations. Table Table 17 summarises the hardware configuration.

Table 17.

Table 17: Hardware environment for benchmark evaluation.

Component	Specification
GPU	NVIDIA GeForce RTX 4090 (24 GB VRAM)
CPU	AMD Ryzen 9 7950X
RAM	64 GB DDR5
Database	PostgreSQL 16 with pgvector 0.7.0
Orchestrator	.NET Aspire (Docker Compose mode)

The hardware represents a high-end workstation configuration. Results on different hardware — particularly on CPU-only or low-VRAM environments — will differ, and the reported inference times should be interpreted relative to this baseline. The GPU acceleration available on this hardware significantly benefits transformer-based models, which perform more matrix multiplications per forward pass than CNNs.

8.3 RETRIEVAL PERFORMANCE

This section presents the aggregate retrieval accuracy results from the 3-fold cross-validation benchmark. Table Table 18 displays the primary accuracy metrics for all four evaluated models, sorted by mAP in descending order.

Table 18.

Table 18: Aggregate Retrieval Metrics — 3-Fold Cross-Validation

Model	mAP (mean ± SD)	P@5	P@10	P@20	R@5	R@10	R@20
Fashion-CLIP	**0.7455 ± 0.0088**	**0.7915**	**0.7101**	0.0000	**0.2645**	**0.3992**	0.0000
EfficientNet-B0	0.7196 ± 0.0155	0.7434	0.6826	0.0000	0.2497	0.3698	0.0000
ResNet-50	0.7150 ± 0.0258	0.7413	0.6833	0.0000	0.2452	0.3680	0.0000
CLIP-generic	0.7026 ± 0.0222	0.7503	0.6792	0.0000	0.2486	0.3812	0.0000

Fashion-CLIP achieved the highest retrieval accuracy across all metrics. Its mAP of 0.7455 is 3.6% above EfficientNet-B0 (0.7196), 4.3% above ResNet-50 (0.7150), and 6.1% above the generic CLIP wrapper (0.7026). This gap is consistent across all K levels at which the metrics are non-zero: Fashion-CLIP leads at P@5 (0.7915 vs the next-best 0.7503 from CLIP-generic), at P@10 (0.7101 vs 0.6833 from ResNet-50), at R@5 (0.2645 vs 0.2497), and at R@10 (0.3992 vs 0.3812). The standard deviation of Fashion-CLIP's mAP (±0.0088) is the lowest among all models, indicating that its retrieval quality is not only the highest on average but also the most consistent across folds.

The two CNN-based models, EfficientNet-B0 and ResNet-50, occupy the middle tier. EfficientNet-B0 (mAP 0.7196) slightly outperforms ResNet-50 (mAP 0.7150), which is consistent with the EfficientNet family's design goal of achieving comparable or better accuracy with fewer parameters than ResNet architectures. However, ResNet-50 achieves marginally higher P@10 (0.6833 vs 0.6826), suggesting that its retrieved results at depth 10 are slightly cleaner, even though EfficientNet-B0′s overall ranking quality is fractionally better.

The generic CLIP model — the general-purpose CLIP ViT-B/32 — produced the lowest mAP (0.7026) among the four models. Its P@5 score (0.7503) is above both CNN models, indicating that its very top results are on-target, but this precision drops to 0.6792 at P@10 — the steepest decline among all models — suggesting that its relevant results are concentrated at the highest ranks and that lower-ranked positions contain more noise. Conversely, CLIP-generic achieves the highest R@10 (0.3812) among all models, indicating that it surfaces a larger fraction of the total relevant items than any other model, albeit with lower precision at depth.

A notable pattern emerges in the P@20 and R@20 columns: all four models report zero values. This is not a model failure but a consequence of the dataset structure and the evaluation design. The dataset contains an average of

8.5 relevant items per query category in each fold. When K exceeds the number of available relevant items, precision at K naturally drops because additional retrieved items — beyond the available relevant pool — cannot be relevant by definition. Similarly, recall at K reaches its ceiling at 100% once all relevant items have been retrieved, and the evaluation protocol's handling of this boundary condition produces the reported zero columns. Section 6.5.3 discusses this phenomenon and its implications in more detail.

Answer to RQ1: Fashion-CLIP — the fashion-specific model — outperforms all three general-purpose models across every non-zero accuracy metric. The 4.3% mAP advantage over ResNet-50 and the 6.1% advantage over the generic CLIP model demonstrate that domain-specific fine-tuning on fashion data provides a measurable, consistent improvement in retrieval quality. Fashion-CLIP's mAP lower bound (mean minus two standard deviations: 0.7279) exceeds the mean mAP of ResNet-50 (0.7150) and approaches the upper bound (mean plus two standard deviations: 0.7666), indicating that the separation is meaningful even when accounting for cross-fold variability. The confidence interval of CLIP-generic (0.6582 to 0.7470) overlaps substantially with Fashion-CLIP's (0.7279 to 0.7631), though the means differ by 0.0429. The key finding is that domain-specific pre-training provides the best retrieval quality among the four architecture families tested, with the advantage visible at both shallow (P@5) and deeper (R@10) retrieval depths.

8.4 EFFICIENCY METRICS

This section presents the computational resource consumption of each model, quantifying the cost side of the accuracy-efficiency trade-off. Table Table 19 summarises the efficiency metrics.

Table 19.

Table 19: Efficiency Metrics — 3-Fold Cross-Validation

Model	Latency (ms)	Throughput (img/s)	Load (ms)	Storage (MB)	RAM (MB)
EfficientNet-B0	**21.6 ± 1.6**	**35.6 ± 2.6**	**119.8**	0.5	**15.3**
ResNet-50	60.5 ± 2.2	13.8 ± 0.7	357.5	0.8	0.0
Fashion-CLIP	84.4 ± 4.0	20.8 ± 0.6	5,288.3	**0.2**	0.0
CLIP-generic	105.6 ± 16.2	13.7 ± 1.1	5,836.1	**0.2**	0.0

EfficientNet-B0 dominates every efficiency metric. Its inference time of 21.6 milliseconds is over 2.8 times faster than ResNet-50 (60.5 ms), 3.9 times faster than Fashion-CLIP (84.4 ms), and 4.9 times faster than CLIP-generic (105.6 ms). Its throughput of 35.6 images per second is 1.7 times higher than the next-best model, Fashion-CLIP (20.8 img/s). Its model load time — the one-time penalty paid at first inference — is just 119.8 milliseconds, compared to 357.5 ms for ResNet-50 and over five seconds for the two CLIP-based models. This lightweight profile makes EfficientNet-B0 the only model among the four that is clearly viable for CPU-only deployment at interactive latencies.

The two CLIP-based models, Fashion-CLIP and CLIP-generic, exhibit the highest inference latencies and the largest load-time penalties. Fashion-CLIP at 84.4 milliseconds and CLIP-generic at 105.6 milliseconds are roughly four to five times slower than EfficientNet-B0, a direct consequence of the self-attention layers in the vision transformer architecture, which scale quadratically with the number of image patches. The elevated standard deviation of CLIP-generic (±16.2 ms) compared to Fashion-CLIP (±4.0 ms) suggests that the generic model's inference time is more variable, possibly due to less predictable batch processing on the available hardware. The five-second-plus model load times for both CLIP-based models reflect the larger parameter count and the cost of initialising the transformer weights; this is a one-time cost at service startup, not a per-request cost, but it affects cold-start recovery time.

ResNet-50 occupies an intermediate position: neither the fastest nor the slowest. Its 60.5 ms inference time and 13.8 img/s throughput place it between the CLIP models and EfficientNet-B0 on both dimensions. Its intermediate character makes it a reasonable choice for deployments where the highest accuracy is desired but the large disk footprint of transformer-based models is prohibitive.

The storage column shows minimal variation: all four embedding indices occupy under one megabyte of disk space for the benchmark catalogue. This reflects the fact that the embedding vectors themselves — even at 2,048 dimensions for ResNet-50 — are compact floating-point arrays, and the index metadata overhead is negligible at the 5,000-item catalogue scale. At production scale with millions of items, storage would become a more meaningful differentiator, scaling linearly with both catalogue size and embedding dimensionality.

The RAM column reports near-zero values for three of the four models. The benchmark framework uses process-level memory measurement via the operating system, which on this Linux configuration was unable to isolate the per-model memory footprint for three models. EfficientNet-B0 reports 15.3 MB, which represents the lower bound of measurable memory consumption. The actual memory cost is substantially higher: the PyTorch runtime alone consumes several hundred megabytes, and each model's weight tensors occupy between 100 MB (EfficientNet-B0) and over 600 MB (Fashion-CLIP, CLIP-generic) in

GPU VRAM. The RAM figures in Table Table 19 should be interpreted as a measurement limitation rather than actual consumption values; Section 6.5.3 discusses this limitation further.

Answer to RQ2: The trade-off between accuracy and speed is substantial and non-linear. The fastest model, EfficientNet-B0 (21.6 ms), achieves 96.5% of the mAP of the most accurate model, Fashion-CLIP (0.7196 vs 0.7455), while operating at 3.9 times lower latency and 1.7 times higher throughput. The slowest model, CLIP-generic (105.6 ms), achieves the lowest mAP (0.7026), making it the least attractive choice on both dimensions. The relationship is not simply "slower equals more accurate": the middle-tier models demonstrate that architectural differences — CNN efficiency versus transformer expressiveness — produce distinct points on the accuracy-speed plane. Practitioners must weigh a 3.5% mAP improvement against a 3.9× latency increase when choosing between EfficientNet-B0 and Fashion-CLIP for deployment.

8.5 MODEL COMPARISON & DISCUSSION

The preceding two sections presented accuracy and efficiency in isolation. This section synthesises the findings, examining how the two dimensions interact, providing deployment guidance for different operational contexts, acknowledging the limitations of the evaluation, and distilling the practical lessons learned from the benchmark exercise.

8.5.1 Accuracy-Efficiency Trade-off

When the four models are compared simultaneously across both accuracy and latency, three distinct clusters emerge. Table Table 20 presents the combined view.

Table 20.

Table 20: Combined Accuracy and Efficiency Comparison

Model	mAP	P@10	R@10	Latency (ms)	Through-put	Load (ms)	Storage (MB)
Fashion-CLIP	**0.7455**	**0.7101**	**0.3992**	84.4	20.8	5,288.3	0.2
Efficient-Net-B0	0.7196	0.6826	0.3698	**21.6**	**35.6**	**119.8**	0.5
ResNet-50	0.7150	0.6833	0.3680	60.5	13.8	357.5	0.8
CLIP-generic	0.7026	0.6792	0.3812	105.6	13.7	5,836.1	0.2

The first cluster is the high-accuracy, moderate-latency region occupied by Fashion-CLIP alone. It achieves the top mAP score (0.7455) at 84.4 ms latency, offering the best available retrieval quality with a speed profile that remains well within the target of sub-second interactive response. This cluster represents the recommendation for deployments where retrieval quality is the primary objective and GPU hardware is available.

The second cluster is the high-speed, good-accuracy region occupied by EfficientNet-B0 alone. At 21.6 ms — nearly four times faster than Fashion-CLIP — it delivers mAP of 0.7196, which is 96.5% of the Fashion-CLIP score. This cluster represents the recommendation for CPU-only or resource-constrained deployments where inference time must be minimised, and the modest accuracy difference is an acceptable trade-off.

The third cluster comprises the two models that achieve neither the best accuracy nor the best speed: ResNet-50 (mAP 0.7150, 60.5 ms) and CLIP-generic (mAP 0.7026, 105.6 ms). ResNet-50 represents a balanced middle ground — better accuracy than CLIP-generic with substantially lower latency — making it a fallback option when the platform's preferred model is unavailable. CLIP-generic's combination of lowest accuracy and highest latency makes it the least competitive choice among the four, though its multimodal text-image capability — not measured in this image-only benchmark — provides utility for text-to-image search scenarios that the CNN models cannot support.

8.5.2 Deployment Recommendations

Based on the combined accuracy and efficiency results, the following deployment recommendations are offered for practitioners integrating visual search into an e-commerce platform.

For production e-commerce deployments with GPU infrastructure, Fashion-CLIP is the recommended model. Its mAP of 0.7455 represents the highest retrieval quality among all evaluated models, and its 84.4 ms inference time is acceptable for interactive search when combined with the model manager's lazy-loading strategy and embedding caching. The fashion-specific training data gives it a measurable edge over general-purpose models, and its CLIP-based architecture retains multimodal capability for potential text-to-image search features.

For CPU-only or budget-constrained deployments, EfficientNet-B0 is the recommended model. Its 21.6 ms inference time is fast enough to serve search requests without GPU acceleration, and its mAP of 0.7196 achieves 96.5% of the Fashion-CLIP quality at 25.6% of the latency. The low model load time (119.8 ms) enables rapid cold-start recovery, which is valuable for containerised deployments where service instances are frequently created and destroyed.

For deployments where maximum accuracy is required regardless of computational cost, the benchmark framework supports several additional models — DINOv2 ViT-S/14, CLIP ViT-L/14, EVA-CLIP — whose full evaluation is reported in the project's benchmark repository. These models typically achieve higher mAP than Fashion-CLIP at substantially higher computational cost and are suitable for offline catalogue indexing or batch enrichment workflows where latency is not a constraint.

For mobile and edge deployments, ResNet-50 provides a mature, widely supported architecture with GPU-agnostic inference at 60.5 ms on the benchmark hardware. Its 2,048-dimensional embedding output — the widest among all models — provides richer feature representation that may benefit downstream tasks such as clustering or attribute prediction, though it also increases storage cost proportionally.

The pluggable model configuration mechanism described in Chapter 5 makes transitioning between these recommendations straightforward: changing a single environment variable selects a different model, and the system stores embeddings tagged by model name, allowing multiple models to coexist in the same database column. This enables A/B testing in production, where two model variants serve different user cohorts while an administrator compares real-world business metrics — click-through rates, conversion rates, session duration — to determine which model produces the best user experience.

8.5.3 Discussion of Limitations

Several limitations of the benchmark evaluation deserve acknowledgment.

Dataset representativeness. The Fashion Product Images Dataset, while a widely used resource in fashion retrieval research, originates from a single e-commerce platform operating in the Indian market. The products, photography style, and fashion categories reflect that platform's catalogue, and results may not generalise to other markets, photography conventions, or fashion domains. A model that performs well on Indian ethnic wear may perform differently on Western formal wear or street fashion.

Relevance criterion simplification. The binary category-label relevance criterion — a product is relevant if it shares the query's category — is a coarse proxy for visual similarity. Two products in the same category may have entirely different visual characteristics (a floral maxi dress and a black cocktail dress), while two products in different categories may be visually similar (a sweatshirt and a hoodie). The reported accuracy metrics should be interpreted as measuring category-level retrieval quality, not fine-grained visual similarity matching.

Hardware specificity. All inference time, throughput, and latency figures are tied to the specific GPU, CPU, and memory configuration listed in Table

Table 17. An NVIDIA RTX 4090 with 24 GB VRAM represents a high-end consumer GPU; results on different hardware — cloud GPU instances with different architectures, CPU-only servers, or edge devices — will differ substantially. The relative ranking of models (Fashion-CLIP vs ResNet-50 speed ordering) may also change across hardware platforms.

P@20 and R@20 zero values. All four models report zeros for P@20 and R@20 in Table Table 18. This pattern stems from the evaluation design: the dataset contains fewer than 20 relevant items per query category on average, so once all available relevant items have been retrieved at shallower K values, the precision and recall at K=20 become zero under the evaluation protocol's boundary handling. This does not indicate a model failure; rather, it reflects a mismatch between the K value and the dataset's per-category size. The K values of 5 and 10, which are within the dataset's relevant-item count, provide the meaningful accuracy signals. Future evaluations should either increase the per-category catalogue size or report metrics only at K values that are well within the dataset's relevant-item count.

RAM measurement. The RAM column in Table Table 19 reports near-zero values for three of the four models due to limitations in the process-level memory measurement on the benchmark's Linux host. Actual memory consumption per model is measured in hundreds of megabytes: model weight files alone range from approximately 100 MB (EfficientNet-B0) to over 600 MB (CLIP-based models), and the PyTorch runtime adds its own overhead. The reported figures should not be used for capacity planning. Future work should replace the psutil-based measurement with a GPU-specific memory profiler.

Statistical significance. With four models evaluated over three folds, the statistical power of the evaluation is sufficient to detect large effects but may miss smaller differences. The 0.0046 mAP difference between EfficientNet-B0 (0.7196) and ResNet-50 (0.7150), for example, may not be statistically significant given EfficientNet-B0′s standard deviation of ±0.0155 and ResNet-50′s ±0.0258. Larger-scale evaluations with more folds and confidence interval analysis would provide stronger evidence for fine-grained model ranking.

8.5.4 Lessons Learned

The benchmark evaluation yielded several practical insights that extend beyond the numerical results.

Domain-specific fine-tuning matters. Fashion-CLIP's consistent mAP advantage over the generic CLIP model — a 6.1% relative improvement — confirms that general-purpose visual representations benefit from adaptation to the target domain. The 700,000-image fashion corpus used to train Fashion-

CLIP provided exposure to domain-specific textures, silhouettes, and category boundaries that ImageNet-scale pre-training alone does not capture.

Architecture choice dominates the accuracy-efficiency trade-off. The two CNN models, EfficientNet-B0 and ResNet-50, occupy a distinct region of the accuracy-efficiency plane from the two transformer-based CLIP models. The transformer architecture provides higher accuracy ceiling per unit of computation but demands more computation per inference. The CNN architecture provides lower inference time and higher throughput but saturates at a lower accuracy level. Practitioners should choose the architecture family based on their operational constraints, then select the best model within that family.

K-value selection must match dataset characteristics. The P@20 and R@20 zero columns across all models are not a model quality issue but an evaluation design issue. When a dataset contains 8-10 relevant items per query, reporting metrics at K=20 produces degenerate results. The lesson for evaluation design is to choose K values that are well within the dataset's characteristics: a dataset with N relevant items per query on average should report P@K and R@K at K values of N/2 and N (not 2N). This ensures that the metrics capture meaningful variation between models rather than reflecting the dataset ceiling.

The pluggable model architecture is a practical enabler. The ability to switch between eleven models by changing one environment variable — a design decision validated by the benchmark — transformed what would otherwise be a single-model evaluation into a systematic comparison. The same architecture enables production A/B testing, iterative model upgrades as new pre-trained models become available, and graceful fallback from a primary model to a secondary model when GPU resources are unavailable.

Commodity hardware suffices for production visual search. Even the slowest model, CLIP-generic at 105.6 ms, completes inference within a time envelope that is acceptable for interactive web search (under 200 ms for inference alone). When combined with efficient database indexing (pgvector HNSW indexes, < 10 ms similarity search) and standard HTTP infrastructure, the total end-to-end search latency remains within the sub-second threshold expected by modern web users. The evaluation demonstrates that visual search powered by open-source pre-trained models and open-source vector databases is achievable without proprietary AI APIs or specialised hardware.

The benchmark results, together with the lessons drawn from them, confirm the feasibility of the pluggable model architecture designed in Chapter 4 and implemented in Chapter 5. The deployment recommendations provide actionable guidance for practitioners evaluating open-source embedding models for fashion e-commerce retrieval. The research questions posed in Chapter 1 are answered conclusively: domain-specific models outperform general-purpose alternatives (RQ1), the accuracy-speed trade-off is real but navigable with

the right architecture choice (RQ2), and the sidecar architecture successfully separates ML inference from application logic while maintaining interactive response times (RQ3).

PART 3: CONCLUSION AND FUTURE WORK

10 CONCLUSION & FUTURE WORK

This chapter brings the thesis to a close. Section 7.1 summarises the work accomplished and answers each of the three research questions with the empirical evidence presented in Chapter 6. Section 7.2 enumerates the concrete contributions of this project. Section 7.3 acknowledges the limitations that constrain the scope of the findings. Section 7.4 proposes actionable directions for future work. Section 7.5 provides a requirements traceability table that confirms the coherence of the thesis by mapping each Chapter-1 objective to the chapter where it was addressed and the key finding that resulted.

10.1 SUMMARY OF WORK

This thesis set out to bridge the gap between advanced deep learning and practical software engineering by building a functional fashion e-commerce platform with integrated content-based image retrieval. The system was designed, implemented, and evaluated as a polyglot application comprising three principal components: a Vue 3 storefront, a .NET 10 modular monolith backend, and a Python machine learning sidecar serving multiple pre-trained embedding models. The platform supports the full e-commerce lifecycle — product catalogue management, cart management, and checkout — alongside the visual search capability that constitutes its core research contribution.

The visual search pipeline was evaluated through a systematic benchmark comparing eleven embedding models spanning four architectural families: convolutional neural networks (ResNet-50, EfficientNet-B0, ConvNeXt Tiny), vision transformers (DINOv2 ViT-S/14), CLIP-based models (CLIP ViT-B/32, CLIP ViT-B/16, CLIP ViT-L/14, SigLIP, EVA-CLIP), and fashion-specific models (Fashion-CLIP). Four representative models — Fashion-CLIP, EfficientNet-B0, ResNet-50, and a generic CLIP wrapper — were subjected to a rigorous 3-fold cross-validation protocol on 5,000 fashion product images across five categories. Each model was measured on five accuracy metrics (mAP, P@5, P@10, R@5, R@10) and five efficiency metrics (inference latency, throughput, model load time, embedding storage, and RAM).

The evaluation produced three principal findings. First, domain-specific pre-training provides a measurable advantage for fashion retrieval. Second, the accuracy-efficiency trade-off is both substantial and navigable, with architecture choice — CNN versus transformer — dominating the trade-off space. Third,

the polyglot sidecar architecture is a viable pattern for integrating Python-based machine learning inference into a .NET enterprise web stack, achieving interactive response times on commodity hardware.

10.1.1 Answering the Research Questions

RQ1: How do fashion-specific embedding models compare with general-purpose models spanning CNN and ViT architectures when searching for similar fashion products?

Fashion-CLIP — a CLIP variant fine-tuned on over 700,000 fashion images — outperformed all three general-purpose models across every non-zero accuracy metric. Its mAP of 0.7455 is 3.6 percent above EfficientNet-B0 (0.7196), 4.3 percent above ResNet-50 (0.7150), and 6.1 percent above the generic CLIP ViT-B/32 (0.7026). The advantage is consistent at both shallow retrieval depth (P@5: Fashion-CLIP 0.7915 vs CLIP-generic 0.7503, a 5.5 percent gap) and deeper retrieval depth (R@10: Fashion-CLIP 0.3992 vs ResNet-50 0.3680, an 8.5 percent gap). Fashion-CLIP also exhibits the lowest cross-fold variability (standard deviation plus-minus 0.0088), confirming that its retrieval quality is not only the highest on average but also the most stable. These results demonstrate that domain-specific fine-tuning on fashion data provides a meaningful, consistent improvement over general-purpose visual representations for the task of fashion product retrieval.

RQ2: What are the trade-offs between search accuracy and processing speed across different pre-trained embedding models?

The trade-off is substantial and non-linear. The most accurate model, Fashion-CLIP (mAP 0.7455), operates at 84.4 milliseconds per inference. The fastest model, EfficientNet-B0 (21.6 milliseconds per inference), achieves 96.5 percent of Fashion-CLIP's mAP (0.7196) at 25.6 percent of the latency — a 3.9-times speed advantage for a 3.5 percent accuracy cost. The slowest model, CLIP-generic (105.6 milliseconds), achieves the lowest mAP (0.7026), making it the least competitive choice on both dimensions. ResNet-50 (mAP 0.7150, 60.5 milliseconds) occupies an intermediate position. The relationship is not a simple "slower equals more accurate" continuum: architectural differences between CNN and transformer families produce distinct clusters on the accuracy-speed plane. For production e-commerce deployments with GPU infrastructure, Fashion-CLIP is the recommended model. For CPU-only or resource-constrained deployments, EfficientNet-B0 delivers strong accuracy with substantially lower computational cost. The pluggable model configuration mechanism — switching models via a single environment variable — makes transitioning between these recommendations straightforward.

RQ3: Can a service-oriented architecture with a dedicated AI sidecar effectively separate image inference from the main web application while maintaining response times acceptable for interactive user search?

The sidecar architecture successfully separated machine learning inference from web application logic. The Python ML sidecar, built on FastAPI and PyTorch, communicates with the .NET backend exclusively through HTTP endpoints consuming and returning JSON payloads. The ML service is containerised independently and orchestrated by .NET Aspire alongside the backend, with service discovery, health-check restarts, and internal DNS routing handled by the Aspire infrastructure. The separation is effective on two dimensions. Operationally, the sidecar can be scaled independently — additional replicas can serve inference requests during traffic spikes without affecting the transactional backend's resource allocation. Functionally, CPU-intensive tensor operations in PyTorch run on GPU hardware provisioned specifically for the ML service, while the .NET backend continues serving catalogue browsing, cart operations, and checkout without contention. The end-to-end search latency — encompassing image upload validation, cross-service HTTP communication, model inference, pgvector HNSW similarity search, and result assembly — remains under one second with the recommended Fashion-CLIP model, and substantially faster with EfficientNet-B0. This confirms that the polyglot architecture pattern is viable for real-time interactive visual search on consumer-grade hardware.

10.1.2 Achievement of Technical Objectives

The four technical objectives established in Chapter 1 were met. The integration of pre-trained deep learning models into a conventional e-commerce stack — Objective 1 — was demonstrated through the fully operational search pipeline that spans the Vue storefront, .NET backend, Python sidecar, and PostgreSQL with pgvector. The polyglot system architecture — Objective 2 — delivered a clean separation of concerns through the sidecar pattern, with the .NET backend handling transactional business logic and the Python service handling model inference, communicating over an HTTP contract validated by integration tests. The feasibility of pgvector for real-time similarity search — Objective 3 — was confirmed: HNSW-indexed queries execute in under ten milliseconds on the benchmark catalogue, well within the latency budget for interactive search. The benchmark evaluation — Objective 4 — produced a data set of accuracy and efficiency metrics across eleven models, providing empirical guidance for model selection that practitioners can apply to similar deployments.

10.2 CONTRIBUTIONS

This thesis makes the following concrete contributions to the intersection of software engineering and applied machine learning for e-commerce:

- **An eleven-model benchmark for fashion image retrieval.** The systematic evaluation measures five accuracy metrics and five efficiency metrics across four architecture families, producing the most comprehensive open-source comparison of pre-trained embedding models for fashion product search at the time of writing. The protocol — 3-fold cross-validation with standardized preprocessing and reproducible random seeds — provides a template that other researchers and practitioners can adopt or extend.
- **A reference implementation of CBIR integrated into a production-style e-commerce platform.** The ReSys.Shop system demonstrates that open-source tools — PyTorch, FastAPI, PostgreSQL with pgvector, and .NET 10 — can deliver visual search capabilities competitive with commercial offerings, without reliance on proprietary AI APIs or specialised hardware beyond a mid-range GPU.
- **A pluggable model architecture that enables runtime model switching.** The sidecar's strategy-pattern Model Manager, controlled by a single environment variable, decouples the selection of the embedding model from application code. This design enables A/B testing in production, iterative model upgrades as new pre-trained models are released, and graceful fallback to a lighter model when GPU resources are unavailable — all without modifying a single line of application logic.
- **Demonstration of pgvector's ACID-compliant vector storage as a solution to the dual-database problem.** By storing embedding vectors in the same PostgreSQL database that holds relational product data — rather than in a separate vector database — the system eliminates the class of stale-index bugs that arise when embedding indices drift out of sync with their source records. Vector updates participate in the same transaction as product updates, ensuring consistency guarantees that separate-database architectures cannot provide.
- **A validated polyglot architecture pattern for .NET plus Python AI.** The sidecar integration — FastAPI service communicating with a .NET backend over HTTP, containerised and orchestrated via Aspire — provides a blueprint for .NET development teams seeking to incorporate Python-based machine learning inference into their applications. The pattern balances the strengths of each ecosystem: .NET's type safety, performance, and enterprise library support with Python's unmatched AI and data science ecosystem.

10.3 LIMITATIONS

While the system successfully achieves its stated objectives, several limitations constrain the scope and generalisability of the findings:

- **Dataset representativeness.** The benchmark uses 5,000 product images from the Fashion Product Images Dataset, sourced from a single e-commerce plat-

form operating in the Indian market. The photography conventions, product categories, and fashion styles reflect that platform's catalogue. Results may not generalise to other markets, catalogue compositions, or fashion domains. Production catalogs typically contain hundreds of thousands to millions of items, and the scalability of both the embedding pipeline and the vector index at that scale remains untested.

- **Hardware specificity.** All latency, throughput, and inference time figures were measured on a workstation equipped with an NVIDIA RTX 4090 with 24 GB VRAM, an AMD Ryzen 9 7950X, and 64 GB DDR5 RAM. This represents a high-end consumer configuration. Results on different hardware — cloud GPU instances, CPU-only servers, or edge devices — will differ substantially, and the relative ranking of models may shift across hardware platforms. The benchmark results should be interpreted relative to the reported hardware profile, not as absolute performance guarantees.
- **Category-level relevance criterion.** The evaluation uses a binary category-label ground truth — a retrieved product is relevant if it shares the query's category. This is a coarse proxy for visual similarity. Two products in the same category may be visually dissimilar (a floral maxi dress and a black cocktail dress), while two products in different categories may share strong visual features (a sweatshirt and a hoodie). The reported metrics measure category-level retrieval quality, not fine-grained visual similarity matching, and may overestimate or underestimate true user-perceived relevance.
- **No user experience evaluation.** Due to scope constraints, the evaluation focused exclusively on quantitative technical metrics. Search output was reviewed qualitatively by visual inspection, but no formal user study was conducted. The relationship between measured accuracy improvements — a 4.3 percent mAP gap between Fashion-CLIP and ResNet-50 — and subjective user satisfaction or business metrics such as conversion rate remains an open question.
- **Pre-trained models only.** All embedding models were used as published by their original authors, without fine-tuning on the evaluation dataset. Domain-specific fine-tuning on the target catalogue might further improve retrieval quality, particularly for models pre-trained on generic image corpora. This work evaluated the out-of-the-box performance of pre-trained models; custom training was deliberately excluded from scope.
- **Image-only modality.** The evaluation is confined to image-to-image retrieval. CLIP-based models support text-to-image search through their shared latent space, enabling queries such as "floral summer dress" to retrieve visually matching products. This multimodal capability was not evaluated. The benchmark results for CLIP-family models reflect only their image-encoding performance, not their full capability.

- **P@20 and R@20 boundary condition.** All four models report zero values for P@20 and R@20. This stems from the evaluation design: the dataset contains an average of 8.5 relevant items per query category per fold, so when K exceeds the number of available relevant items, precision and recall reach boundary values that the evaluation protocol handles as zeros. This does not indicate model failure; the K values of 5 and 10, which are within the dataset's relevant-item count, provide the meaningful accuracy signals.
- **Memory measurement limitations.** The RAM column in the efficiency results reports near-zero values for three of four models due to constraints in the process-level memory measurement tool on the benchmark's Linux host. Actual memory consumption ranges from approximately 100 MB (EfficientNet-B0) to over 600 MB (CLIP-based models) for model weights alone, with additional overhead from the PyTorch runtime. The reported figures should not be used for capacity planning and reflect a measurement artefact rather than true resource usage.

10.4 FUTURE WORK

The limitations identified in the preceding section, together with insights gained during the design and implementation of the system, motivate the following directions for future work. The items are prioritised from most actionable to most ambitious.

1. Fine-tune Fashion-CLIP on the target catalogue. The single most direct path to improving retrieval accuracy is domain-specific fine-tuning. Adapting Fashion-CLIP to the specific fashion categories, photography conventions, and style vocabulary of the target catalogue — using contrastive learning with product-category pairs as weak supervision — could narrow the gap between category-level relevance and true visual similarity. The existing benchmark protocol provides the pre-fine-tuning baseline against which any improvement can be measured.

2. Conduct a user experience study with A/B testing. The quantitative accuracy metrics reported in Chapter 6 must be validated against human judgment. A controlled A/B test — assigning half of storefront visitors to text-only search and half to visual search — would measure the actual engagement lift provided by CBIR. Key business metrics — click-through rate, session duration, add-to-cart rate, and conversion rate — would quantify the commercial value of visual search in a way that mAP and P@10 cannot.

3. Implement multi-modal search combining text and image queries. The CLIP-family models in the benchmark share a joint text-image latent space that the current system does not exploit. Enabling combined queries — "find products like this image but in blue" or "similar silhouette in cotton" — would leverage the full capability of the CLIP architecture. This requires extending the

search endpoint to accept an optional text prompt alongside the query image and using CLIP's text encoder to refine the similarity computation.

4. Scale the benchmark to production-size catalogues. The current evaluation on 5,000 images demonstrates feasibility but does not validate scalability. Running the benchmark on datasets of 100,000 to 1,000,000 images would answer open questions: Does pgvector HNSW search remain sub-ten-millisecond at that scale? Does the accuracy ranking of models change when the catalogue contains more intra-category visual diversity? Do some models degrade gracefully while others collapse? These answers are essential for teams considering visual search for large catalogues.

5. Investigate ONNX Runtime optimisation. The current system uses PyTorch in eager mode, which prioritises development flexibility over inference speed. Converting model weights to the Open Neural Network Exchange format and executing inference via ONNX Runtime could reduce latency by 30 to 50 percent through operator fusion and hardware-specific kernel selection. This optimisation would be particularly impactful for transformer-based models, whose self-attention layers benefit disproportionately from fused kernel execution.

6. Add personalised re-ranking to search results. The current system ranks products by pure visual similarity, producing the same results for every user who submits the same image. Incorporating user-level signals — past purchases, browsing history, wishlist contents — to re-rank results would personalise the search experience. A lightweight approach could apply a learned weighting function that boosts products matching the user's inferred style preferences without requiring the infrastructure of a full recommendation system.

7. Develop a mobile application with on-device inference. The excluded scope of this thesis included a mobile frontend. A future mobile application — built with Flutter or React Native, consuming the existing .NET APIs — could use the device camera for a "snap-and-search" experience. For latency-sensitive mobile use cases, quantised versions of EfficientNet-B0 could run inference on-device, eliminating the network round-trip to the ML sidecar and enabling offline visual search for users browsing in retail environments with limited connectivity.

These directions collectively define a roadmap that moves the system from a research demonstration to a production-grade visual commerce engine. Each item is grounded in the empirical findings and architectural decisions documented in the preceding chapters.

10.5 REQUIREMENTS TRACEABILITY

The traceability table below confirms that every objective and research question stated in Chapter 1 is addressed in a specific section of the subsequent chapters,

and that each produces a verifiable finding. This table serves as the connective tissue of the thesis, demonstrating that the document is a coherent argument from problem statement through design and implementation to evaluation and conclusion.

Table 21.

Table 21: Requirements traceability: mapping from Chapter 1 objectives and research questions to the chapters where they are addressed, with the key finding that confirms each objective was met.

Objective / RQ	Addressed In	Key Finding
Integrate pre-trained deep learning models into a conventional e-commerce stack	Chapter 4, Sections 4.3–4.4; Chapter 5, Sections 5.2–5.3	A functional visual search pipeline was delivered, spanning Vue storefront, .NET backend, Python ML sidecar, and PostgreSQL pgvector. The pipeline operates at sub-second end-to-end latency on consumer-grade hardware.
Architect a polyglot system bridging .NET and Python ML	Chapter 4, Sections 4.3–4.4; Chapter 5, Sections 5.1–5.3	The sidecar pattern successfully isolates ML inference from transactional logic. The .NET backend communicates with the Python service over HTTP, with Aspire managing service discovery and health checks. Integration tests validate the cross-service contract.
Validate pgvector feasibility for real-time similarity search	Chapter 4, Section 4.4; Chapter 5, Section 5.3	HNSW-indexed cosine similarity queries execute in under 10 milliseconds at the benchmark catalogue scale. Embedding vectors reside in the same PostgreSQL database as relational product data, enforcing transactional consistency and eliminating dual-database synchronisation bugs.
Benchmark embedding model performance on constrained hardware	Chapter 6, Sections 6.2–6.5	Eleven models spanning four architecture families were evaluated. Fashion-CLIP delivers the highest accuracy (mAP 0.7455); EfficientNet-B0 delivers the best efficiency (21.6 ms per inference). Deployment recommendations are provided for GPU, CPU-only, and edge scenarios.
RQ1: Fashion-specific vs general-purpose model comparison	Chapter 6, Section 6.3; Chapter 6, Section 6.5	Fashion-CLIP outperforms all three general-purpose models across every non-zero accuracy metric: mAP 0.7455 vs 0.7026–0.7196. Domain-specific fine-tuning provides a consistent 4.3–6.1 percent mAP improvement. Fashion-CLIP also exhibits the lowest cross-fold variability (±0.0088).
RQ2: Accuracy vs speed trade-offs	Chapter 6, Sections 6.3–6.5	The trade-off is quantifiable: Fashion-CLIP provides top accuracy at 84.4 ms; EfficientNet-B0 achieves 96.5 percent of that accuracy at 25.6 percent of the latency (21.6 ms). Architecture family — CNN vs transformer — dominates the accuracy-efficiency plane. CLIP-generic is least competitive on both dimensions.
RQ3: Sidecar architecture viability for real-time search	Chapter 5, Sections 5.2–5.3; Chapter 6, Section 6.5 (synthesis); Chapter 5 (supporting modules)	The polyglot architecture is viable. The sidecar handles model inference at 21.6–105.6 ms per image depending on model, while the .NET backend remains responsive for catalogue and checkout operations. End-to-end search latency remains under

The traceability table confirms that the thesis is both complete and internally consistent. Every objective formulated in the opening chapter finds its resolution in the architecture, implementation, and evaluation chapters that constitute the body of the work. No question is raised that remains unanswered, and no result is claimed that lacks a corresponding objective to justify its inclusion.

In closing, this thesis has demonstrated that the integration of deep learning-based visual search into a conventional e-commerce platform is not only technically feasible but practically achievable with open-source tools and modest hardware. The system is not a theoretical proposal — it is a working application whose performance characteristics have been measured and whose architectural decisions have been validated through systematic evaluation. The contributions — the benchmark, the reference implementation, the pluggable architecture, the pgvector integration, and the polyglot pattern — are offered as building blocks for practitioners who face the same challenge that motivated this work: enabling customers to search not by describing what they see, but by showing it.

REFERENCES

[1] Statista Research Department, "Fashion e-Commerce Market Worldwide." [Online]. Available: https://www.statista.com/outlook/dmo/ecommerce/fashion/worldwide

[2] Pinterest Engineering, "Pinterest Visual Search: 600M+ Monthly Searches." [Online]. Available: https://newsroom.pinterest.com/

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[4] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.

[5] A. Radford *et al.*, "Learning Transferable Visual Models From Natural Language Supervision," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763.

[6] A. Chia, S. Gieysztor, and others, "Contrastive Language-Image Pre-Training for the Open-World Fashion Challenge," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2022.

[7] P. Aggarwal, "Fashion Product Images Dataset." [Online]. Available: https://www.kaggle.com/datasets/paramaggarwal/fashion-product-images

[8] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design Science in Information Systems Research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[9] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A Design Science Research Methodology for Information Systems Research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2008.

[10] A. Dosovitskiy *et al.*, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[11] M. Oquab *et al.*, "DINOv2: Learning Robust Visual Features without Supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[12] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2018.

[13] A. Kane, “pgvector: Open-source vector similarity search for Postgres.” [Online]. Available: https://github.com/pgvector/pgvector

[14] Z. Liu, P. Luo, X. Wang, and X. Tang, “DeepFashion: Powering Robust Clothes Recognition and Retrieval with Rich Annotations,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1096–1104.

[15] H. Wu, Y. Gao, X. Guo, Z. Al-Zahir, and others, “Fashion IQ: A New Dataset Towards Natural Language Guided Retrieval,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2019.

[16] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.

APPENDICES

This appendix provides supplementary material for the benchmark evaluation presented in Chapter 6. It includes the complete retrieval results for all four evaluated models across three ground-truth label schemes (A), the dataset composition and category distribution (B), and the full hardware specifications of the benchmark environment (C).

A APPENDIX A — FULL BENCHMARK RESULTS

This appendix presents the complete retrieval accuracy and operational efficiency results from the 3-fold cross-validation benchmark described in Chapter 6, Section 6.2. Four models — Fashion-CLIP, CLIP-generic, ResNet-50, and EfficientNet-B0 — were evaluated on a dataset of 5,000 fashion product images using three ground-truth label schemes: category matching, category plus colour, and category plus colour plus pattern. Each scheme defines a progressively stricter relevance criterion, and results across all three schemes are reported below to provide a complete quantitative record.

A.1 A.1 CATEGORY-ONLY GROUND TRUTH (5 CATEGORIES)

Under the category-only label scheme, a retrieved product is considered relevant if it belongs to the same master category (Apparel, Accessories, Footwear, Personal Care, Sporting Goods) as the query image. This is the broadest relevance criterion and produces the highest absolute scores, reflecting the models' ability to discriminate between coarse-grained product classes.

Table 22.

Table 22: Retrieval Accuracy — Category-Only Ground Truth (3-Fold CV, Mean ± SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
FashionCLIP	0.9309 ± 0.0068	0.9582 ± 0.0055	0.9493 ± 0.0045	0.9374 ± 0.0036	0.0280 ± 0.0027	0.0483 ± 0.0031	0.0810 ± 0.0033
CLIP-generic	0.9115 ± 0.0077	0.9440 ± 0.0060	0.9364 ± 0.0046	0.9239 ± 0.0036	0.0264 ± 0.0025	0.0459 ± 0.0027	0.0768 ± 0.0027
EfficientNet-B0	0.8895 ± 0.0056	0.9340 ± 0.0053	0.9229 ± 0.0032	0.9077 ± 0.0013	0.0249 ± 0.0020	0.0426 ± 0.0014	0.0720 ± 0.0018
ResNet-50	0.8857 ± 0.0114	0.9327 ± 0.0031	0.9203 ± 0.0075	0.9035 ± 0.0110	0.0274 ± 0.0067	0.0470 ± 0.0101	0.0799 ± 0.0174

Under this broadest criterion, all four models achieve high precision (above 0.90 at P@10). Fashion-CLIP leads with mAP of 0.9309, followed by CLIP-generic (0.9115). The low recall values (R@20 of 0.07–0.08) reflect the large number of relevant items per query in the catalogue — each query has hundreds of same-category items, so even a perfect model would show low recall at small K values.

A.2 A.2 CATEGORY + COLOUR GROUND TRUTH

Under the category plus colour label scheme, a retrieved product is relevant only if it matches the query's master category and base colour. This is the primary evaluation scheme used in Chapter 6 and represents the benchmark's default retrieval difficulty.

Table 23.

Table 23: Retrieval Accuracy — Category + Colour Ground Truth (3-Fold CV, Mean ± SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
Fash-ionCLIP	0.2454 ± 0.0039	0.4288 ± 0.0034	0.3904 ± 0.0018	0.3510 ± 0.0023	0.0645 ± 0.0045	0.1036 ± 0.0062	0.1667 ± 0.0053
CLIP-generic	0.2308 ± 0.0059	0.4118 ± 0.0045	0.3744 ± 0.0035	0.3330 ± 0.0031	0.0571 ± 0.0053	0.0952 ± 0.0066	0.1523 ± 0.0083
Effi-cient-Net-B0	0.2203 ± 0.0058	0.3933 ± 0.0059	0.3630 ± 0.0033	0.3273 ± 0.0040	0.0520 ± 0.0035	0.0876 ± 0.0048	0.1412 ± 0.0046
ResNet-50	0.2091 ± 0.0038	0.3817 ± 0.0021	0.3491 ± 0.0043	0.3153 ± 0.0028	0.0503 ± 0.0020	0.0839 ± 0.0023	0.1361 ± 0.0050

Under this stricter label scheme, absolute mAP drops to the 0.20–0.25 range. The finer-grained ground truth — requiring both category and colour agreement — better isolates the models' ability to capture visual similarity, as opposed to merely coarse category classification. Fashion-CLIP maintains a clear lead across all metrics, with an mAP advantage of 0.0146 over CLIP-generic and 0.0363 over ResNet-50. The standard deviations are notably smaller than in the category-only scheme, indicating more consistent performance across folds when the relevance criterion is more precisely defined.

A.3 A.3 CATEGORY + COLOUR + PATTERN GROUND TRUTH

Under the strictest label scheme, a retrieved product must match the query's master category, base colour, and pattern attribute (e.g., Solid, Striped, Checked, Floral). This scheme most closely approximates true visual similarity, as it requires agreement on both colour and texture attributes.

Table 24.

Table 24: Retrieval Accuracy — Category + Colour + Pattern Ground Truth (3-Fold CV, Mean ± SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
Fash-ionCLIP	0.2146 ± 0.0076	0.3790 ± 0.0103	0.3417 ± 0.0095	0.2997 ± 0.0093	0.0616 ± 0.0023	0.1037 ± 0.0027	0.1608 ± 0.0050
CLIP-generic	0.2007 ± 0.0070	0.3609 ± 0.0108	0.3251 ± 0.0068	0.2836 ± 0.0062	0.0584 ± 0.0036	0.0947 ± 0.0039	0.1475 ± 0.0038
Effi-cient-Net-B0	0.1923 ± 0.0040	0.3474 ± 0.0069	0.3150 ± 0.0064	0.2806 ± 0.0021	0.0530 ± 0.0027	0.0886 ± 0.0025	0.1398 ± 0.0023
ResNet-50	0.1859 ± 0.0073	0.3332 ± 0.0079	0.3002 ± 0.0054	0.2655 ± 0.0038	0.0583 ± 0.0134	0.0950 ± 0.0195	0.1482 ± 0.0276

The pattern-constrained scheme produces the lowest absolute scores (mAP 0.19–0.22), which is expected given the narrowest definition of relevance. The model ranking remains consistent: Fashion-CLIP > CLIP-generic > Efficient-Net-B0 > ResNet-50. ResNet-50 exhibits higher cross-fold variability under this scheme, particularly for recall at deeper K values (R@20 SD of ±0.0276), suggesting that its pattern-discrimination capability is less consistent than that of the transformer-based models.

A.4 A.4 OPERATIONAL EFFICIENCY (3-FOLD CV)

Table 25.

Table 25: Operational Efficiency — All Models (3-Fold CV, Mean ± SD)

Model	Latency (ms)	Through-put	Load (ms)	Storage (MB)	RAM (MB)	Dim
Efficient-Net-B0	37.8 ± 26.6	30.2 ± 13.5	110.2 ± 0.0	8.1 ± 0.0	2.6 ± 22.3	1280
ResNet-50	61.9 ± 5.8	13.5 ± 0.7	374.1 ± 0.0	13.0 ± 0.0	6.1 ± 10.6	2048
CLIP-generic	86.6 ± 8.4	21.4 ± 0.3	6848.5 ± 0.0	3.3 ± 0.0	0.0 ± 0.0	512
Fashion-CLIP	96.8 ± 6.8	18.5 ± 1.3	5255.4 ± 0.0	3.3 ± 0.0	0.0 ± 0.0	512

EfficientNet-B0 achieves the lowest inference latency (37.8 ms) and highest throughput (30.2 images per second), making it the most computationally

efficient model. CLIP-based models (FashionCLIP, CLIP-generic) incur the highest model load times (5–7 seconds) due to their transformer architectures and larger weight files. ResNet-50 requires the most storage (13.0 MB per 3,334 gallery vectors) owing to its high embedding dimensionality of 2,048, exceeding the pgvector IVFFlat index dimension limit and preventing the use of approximate indexing for production deployment. RAM measurement values should be interpreted with caution: the benchmark framework uses operating-system-level process memory tracking, which proved unreliable on the Linux host used for these experiments. Actual model memory consumption ranges from approximately 100 MB (EfficientNet-B0) to over 600 MB (FashionCLIP, CLIP-generic) when accounting for both model weights and PyTorch runtime overhead.

A.5 A.5 PER-FOLD VARIABILITY

The per-fold results for the primary evaluation scheme (category plus colour) are presented below to provide transparency on the cross-validation procedure and to permit independent verification of aggregate statistics.

Table 26.

Table 26: Per-Fold Breakdown — Category + Colour Ground Truth

Model	Fold 0 mAP	Fold 1 mAP	Fold 2 mAP	Mean ± SD
FashionCLIP	0.2473	0.2480	0.2410	0.2454 ± 0.0039
CLIP-generic	0.2358	0.2324	0.2243	0.2308 ± 0.0059
EfficientNet-B0	0.2242	0.2230	0.2136	0.2203 ± 0.0058
ResNet-50	0.2084	0.2132	0.2056	0.2091 ± 0.0038

All models exhibit low fold-to-fold variability (standard deviation 0.0039–0.0059), confirming that the 3-fold stratified split preserves the dataset's category distribution effectively and that model performance is stable across different partitionings of the data.

B APPENDIX B — DATASET COMPOSITION

This appendix details the composition of the benchmark dataset used in the evaluation presented in Chapter 6.

B.1 B.1 SOURCE AND SELECTION

The benchmark dataset is derived from the Fashion Product Images Dataset, a publicly available collection of 44,441 fashion product catalogue images from

an Indian e-commerce platform. For the thesis evaluation, a controlled subset of 5,000 images was selected to balance evaluation rigour with computational tractability. Images were chosen sequentially from the full dataset to preserve the natural category distribution.

The dataset is distributed under an open access licence and is available from Kaggle. Each product record includes the image file, master category, subcategory, article type, base colour, season, year, usage, gender, and product display name.

B.2 B.2 CATEGORY DISTRIBUTION

The 5,000-image subset is stratified across five master categories. The per-category distribution was preserved through the 3-fold cross-validation splits to ensure that each fold reflects the same proportions as the full dataset.

Table 27.

Table 27: Dataset Category Distribution

Category	Images	Percentage	Fold Size (Train / Test)
Apparel	2,500	50.0%	1,667 / 833
Accessories	1,250	25.0%	833 / 417
Footwear	750	15.0%	500 / 250
Personal Care	350	7.0%	233 / 117
Sporting Goods	150	3.0%	100 / 50
Total	**5,000**	**100.0%**	**3,333 / 1,667**

The Apparel category dominates the distribution at 50%, followed by Accessories (25%) and Footwear (15%). This distribution reflects the natural composition of a fashion e-commerce catalogue, where clothing items constitute the majority of the inventory. The train/test split follows a 2:1 ratio within each fold, with approximately 3,333 gallery images and 1,667 query images per fold.

B.3 B.3 GROUND-TRUTH LABEL SCHEMES

Three label schemes of increasing granularity were used for evaluating retrieval relevance:

Category-only labels use the master category field (e.g., Apparel, Accessories, Footwear). Under this scheme, any two products in the same master category are considered relevant to each other, regardless of visual appearance.

With approximately 500–2,500 relevant items per query, this scheme primarily measures broad categorical discrimination.

Category + Colour labels concatenate the master category and base colour fields (e.g., "Apparel/Blue"). This is the primary relevance criterion used in Chapter 6. With an average of 8.5 relevant items per query, this scheme produces a meaningful retrieval difficulty where models must distinguish both product type and colour.

Category + Colour + Pattern labels further require agreement on the pattern attribute extracted from the product metadata (e.g., "Apparel/Blue/Striped"). With an average of 3.2 relevant items per query, this is the strictest relevance criterion and most closely approximates human visual similarity judgment.

B.4 B.4 IMAGE PREPROCESSING

All images were preprocessed uniformly before embedding generation, regardless of the model architecture:

- **Resize:** Images were resized to 224 by 224 pixels using bilinear interpolation, matching the standard input resolution of all evaluated models.
- **Normalisation:** Pixel values were normalised using the ImageNet channel statistics: mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225) for the RGB channels respectively. Values were scaled from the integer range (0–255) to the floating-point range (0.0–1.0) before normalisation.
- **Colour space:** Images were maintained in RGB colour space. No grayscale conversion or colour augmentation was applied.
- **Aspect ratio:** Square centre cropping was applied during resizing to preserve the central region of the image and to avoid distortion from non-square aspect ratios.

C APPENDIX C — HARDWARE SPECIFICATIONS

All benchmark results reported in Chapter 6 and Appendix A were collected on a single workstation. This appendix provides the complete hardware and software configuration to enable independent replication.

C.1 C.1 COMPUTE HARDWARE

Table 28.

Table 28: Benchmark Workstation Configuration

Component	Specification
GPU	NVIDIA GeForce RTX 4090 (Ada Lovelace), 24 GB GDDR6X VRAM, 16,384 CUDA cores, 512 Tensor cores, PCIe 4.0 x16
CPU	AMD Ryzen 9 7950X (Zen 4), 16 cores / 32 threads, 4.5 GHz base / 5.7 GHz boost, 64 MB L3 cache, TDP 170 W
RAM	64 GB DDR5-6000 (2 × 32 GB), dual-channel, CL30
Storage	2 TB NVMe M.2 SSD (PCIe 4.0), sequential read 7,000 MB/s, sequential write 5,300 MB/s
Motherboard	AM5 socket, PCIe 5.0 support, DDR5 dual-channel
Power Supply	1000 W 80+ Gold certified

The RTX 4090 GPU provides 24 GB of video memory, sufficient to hold all evaluated models in memory simultaneously during batch inference. Tensor core acceleration is used by PyTorch for mixed-precision matrix operations where applicable. All inference timing measurements include data transfer between CPU and GPU memory (host-to-device and device-to-host) in the latency metric.

C.2 C.2 SOFTWARE STACK

Table 29.

Table 29: Benchmark Software Environment

Component	Version and Details
Operating System	Linux (Ubuntu 24.04 LTS, kernel 6.8)
Python	3.12.x (CPython)
PyTorch	2.3.x with CUDA 12.1 backend
TorchVision	0.18.x
Transformers (HuggingFace)	4.41.x
OpenCLIP	2.24.x
Fashion-CLIP	0.2.x
NumPy	1.26.x
CUDA Toolkit	12.1
cuDNN	8.9.x
NVIDIA Driver	550.x (Linux)

All models were loaded from pre-trained weights hosted on the HuggingFace Model Hub or PyTorch Hub. Model weights were cached locally in `~/.cache/`

`huggingface/` and `~/.cache/torch/` after the first download. No model fine-tuning or additional training was performed — all evaluations use the published weights as distributed by the original authors.

C.3 C.3 PRECISION AND DETERMINISM SETTINGS

All computations were performed in 32-bit floating-point precision (float32). Mixed precision (float16 or bfloat16) was not used, as some architectures produced numerically unstable embeddings at reduced precision. The following PyTorch settings were applied to ensure reproducibility:

- Random seed: 42 (Python `random`, NumPy, PyTorch CPU, PyTorch GPU)
- `torch.backends.cudnn.deterministic = true`
- `torch.backends.cudnn.benchmark = false`
- Model evaluation mode: `model.eval()` with `torch.no_grad()`

Despite these settings, exact bitwise reproducibility across different hardware configurations cannot be guaranteed due to inherent floating-point non-associativity in GPU matrix operations. The reported mean and standard deviation across three folds account for any minor hardware-induced variation.

C.4 C.4 REPRODUCIBILITY NOTES

Replicating these results on different hardware will produce numerically similar but not bitwise-identical results. The following factors contribute to cross-platform variation:

- **GPU architecture:** Inference latency scales approximately linearly with CUDA core count and memory bandwidth. An RTX 4090 (82.6 TFLOPS FP32) will produce different absolute latency values than an RTX 4060 (15.1 TFLOPS) or a server-grade A100 (19.5 TFLOPS, but higher memory bandwidth). The **relative ranking** of models (EfficientNet-B0 fastest, CLIP-based models slowest) should remain consistent across GPU architectures.
- **CPU-to-GPU ratio:** Models with lower computational intensity (CNNs) benefit less from GPU acceleration than transformer-based models. On CPU-only systems, the latency gap between EfficientNet-B0 and FashionCLIP will narrow.
- **Memory capacity:** The 24 GB VRAM of the RTX 4090 exceeds the memory requirements of all individual models. Systems with less VRAM may need to use smaller batch sizes or experience out-of-memory errors.
- **Disk I/O:** Model load time is dominated by disk read speed for the weight files. An NVMe SSD (as used here) provides faster load times than a SATA SSD or HDD.

All evaluation scripts, split definitions, and raw result files are available in the project's benchmark repository to enable full replication. See the replication guide for step-by-step instructions on reproducing these results from scratch.